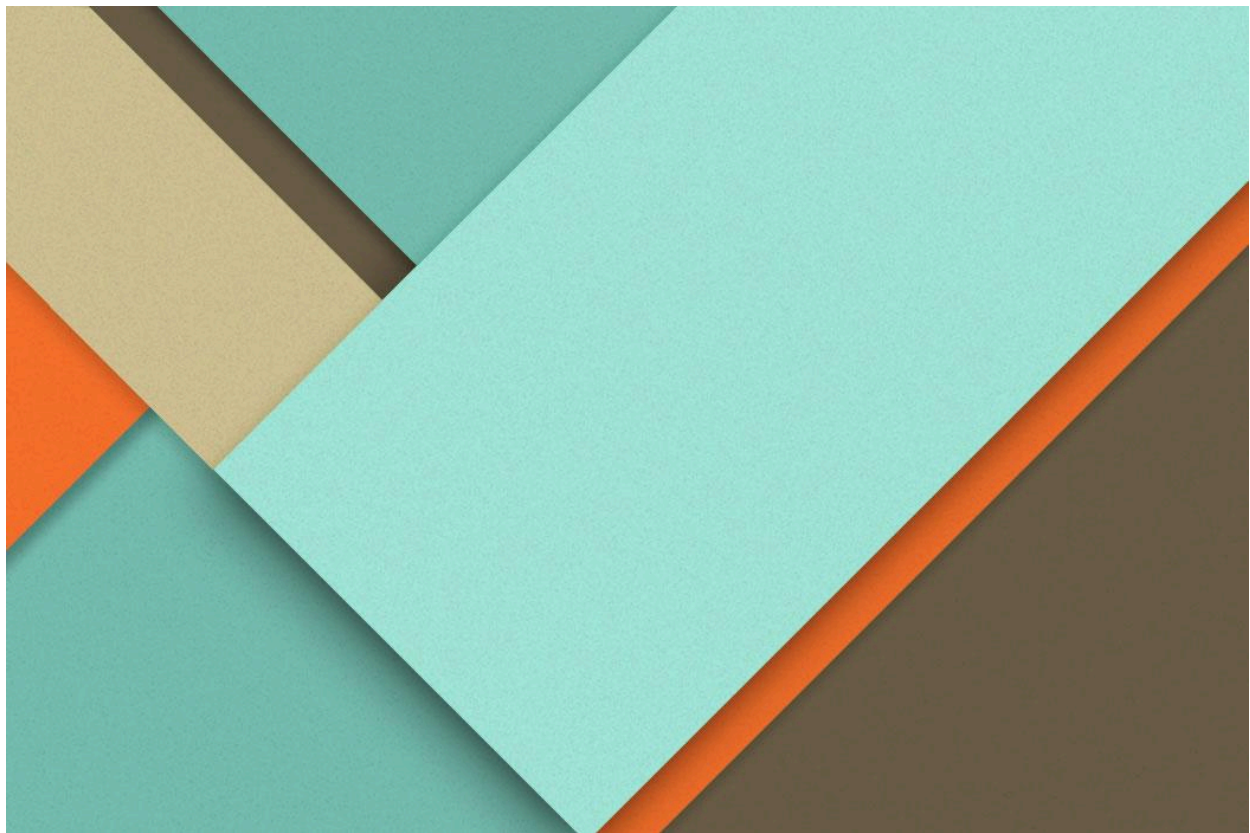




Design and Implementation of a Secure Medical Data Processing System

Ioan Razem - DevOps

Sorin Popa - Security Analyst

Denisa Popa - Project Manager

Victor Nitu - AI Enthusiast

Table of Contents

Project Summary.....	4
Key System Functionalities.....	5
Data Ingestion through MQTT Broker.....	5
Rate Limit.....	11
Secure Data Transmission over MQTT Using mTLS.....	14
Web API Endpoints.....	17
Middleware Stack.....	17
Authentication Flow.....	17
Authorisation Model (RBAC).....	18
Route Table.....	19
Input Validation.....	20
Server Configuration.....	21
Threat / Mitigation Summary.....	21
OCR Sandbox.....	23
Android Mobile Client - Image Capture and Offline Storage.....	27
Database Encryption (PHI Field-Level Encryption).....	32
OCR Confidence Scoring.....	35
Dark Mode Toggle.....	36
Implementation.....	36
Coverage.....	37
Security and Compliance.....	38
CI/CD Evidence Plane Workflow.....	38
CI/CD Pipeline Architecture.....	39
Pipeline 1 — CI Gateway (ci.yml).....	39
Pipeline 2 — Unit Tests & Coverage (tests.yml).....	40
Pipeline 3 — Static Analysis (analysis.yml).....	41
Pipeline 4 — Fuzzing (fuzzing.yml).....	42
Pipeline 5 — Release (release.yml).....	44
AI Agent — Automated PR Security Review.....	45
Branch Protection & Merge Policy.....	46
SARIF Integration & Security Dashboard.....	46
Pipeline Summary Table.....	47
Team Contributions.....	48
Medical Reports.....	49
Report 1 — Medical Opinion Summary (Fitness Breakdown).....	49



Report 2 — Expiration Alerts (Who Expires Next Month).....	50
Report 3 — Anonymized Dataset for Research.....	51
Report 4 — System Performance Metrics.....	52
Report 5 — Control Type Distribution (Statistics Page).....	53
Report 6 — Monthly Activity Comparison.....	54
Summary Table.....	55
OSSF Criticality Score.....	56

Project Summary

The primary objective of this project is the development of an ultra-secure web platform dedicated to processing medical documents and images. Because the application handles highly sensitive Patient Health Information (PHI), its core architecture is built upon a security model governed by Threat Analysis and Risk Assessment (TARA). The platform securely ingests medical files via an MQTT broker, processes them using an Optical Character Recognition (OCR) engine, and generates dynamic analytical reports. Furthermore, the project's lifecycle is governed by an AI-assisted and Security-Optimized CI/CD architecture. This infrastructure utilizes AI agents for autonomous testing and vulnerability remediation while strictly maintaining a deterministic "human-in-the-loop" oversight mechanism.

The repository achieved an OpenSSF Criticality Score of 0.19619, a low result reflecting its status as a new, Go-based academic project rather than a high-adoption public dependency. The score, derived from a two-stage analysis of activity metrics and external metadata via default configuration, highlights low public visibility rather than code quality.

Key System Functionalities

Data Ingestion through MQTT Broker

The MQTT broker is the edge-facing ingestion path of the platform. It addresses the project only channel through which IoT devices (the Android client and Python scripts) push medical images, device registration events, and connection state into the system.

```
PS D:\SS-project> docker compose ps
```

NAME	IMAGE	COMMAND	SERVICE	CREATED	STATUS	PORTS
broker	eclipse-mosquitto:latest	"/docker-entrypoint.s..."	broker	6 minutes ago	Up 6 minutes	0.0.0
0:8883->8883/tcp, [::]:8883->8883/tcp						
go-api	stefandarius/mqtt-ss-api:latest	"/.mqtt"	go-api	6 minutes ago	Up 6 minutes	0.0.0
0:8080->8080/tcp, [::]:8080->8080/tcp						
mongo-db	mongo:latest	"docker-entrypoint.s..."	mongo-db	6 minutes ago	Up 6 minutes	0.0.0
0:27019->27017/tcp, [::]:27019->27017/tcp						
ocr-sandbox	ss-project-ocr-sandbox:local	"uvicorn app.main:ap..."	ocr-sandbox	6 minutes ago	Up 6 minutes (healthy)	9000/
tcp						

Figure 1: Checking all docker containers are running

On the MQTT side, the Mosquitto broker in `broker/mosquitto.conf` was hardened to a TLS-only listener on port 8883 with `require_certificate true` and `use_subject_as_username true`, and the legacy plaintext listener on port 1883 was disabled. Broker-level safety limits were added in the same file: `max_connections 100`, `max_inflight_messages 10`, `max_queued_messages 100`, and `message_size_limit 5242880` (5 MiB). On the application side, `server/broker/payload_limit.go` introduces `RejectOversizedMQTTPayload(topic, payload)`, which enforces a configurable per-message size cap (MQTT_MAX_PAYLOAD_BYTES, default 5 MiB) and emits a structured `SECURITY_EVENT=MQTT_PAYLOAD_SIZE_LIMIT_EXCEEDED` log entry when the limit is exceeded. A sliding-window rate limiter for MQTT was added in `server/broker/rate_limit.go`, exposed through `RejectRateLimitedMQTTMessage(topic)` and configured at runtime via `MQTT_RATE_LIMIT_MESSAGES` and `MQTT_RATE_LIMIT_WINDOW_SECONDS` in `docker-compose.yml` (production values 5 messages per 30 seconds, stricter than the 60/30 code defaults). Both guards are inserted at the very top of all four MQTT message handlers in `server/broker/broker.go` — sensor data ingestion, device registration, device disconnection, and command forwarding — before any business logic, OCR call, or database write can execute.

```
# MQTT over TLS/SSL
listener 8883
message_size_limit 5242880
max_packet_size 10485760
cafile /run/secrets/ca.crt
certfile /run/secrets/server.crt
keyfile /run/secrets/server.key
require_certificate true
use_subject_as_username true
allow_anonymous true
```

Figure 2: Rules for MQTT over TLS/SSL

Directive	Value	Effect
max_connections	100	Hard cap on simultaneous MQTT clients; refused at TCP level
max_inflight_messages	10	Max QoS 1/2 messages in flight per client
max_queued_messages	100	Max queued messages per slow subscriber; memory guard
message_size_limit	5242880 (5 MB)	Per-packet upper bound; oversized publishes are dropped
require_certificate	true	No client certificate means no MQTT handshake
use_subject_as_username	true	Certificate Subject becomes the MQTT identity in broker logs

The broker-level directives in `broker/mosquitto.conf` complement the application-level guards, forming a defence-in-depth boundary that catches abusive traffic at the TCP/TLS layer before a single byte reaches the Go process:

Execution Flow

Every accepted MQTT connection must first complete the mTLS handshake. The broker presents `server.crt` (signed by the CA) and the client presents either `web.crt` (Go API and Python scripts) or the Android-side PKCS12 bundle `android_client.p12`; both sides verify the peer against `ca.crt`. Once the handshake completes, the broker enforces its directive limits (`max_connections` at the TCP layer, `message_size_limit` per packet, inflight and queued counts per session) before any PUBLISH is delivered to a subscriber.

On the Go API side, each delivered message lands in the handler registered in `server/main.go` for the matching topic. The first two operations every handler performs are the application-level guards:

Step	Component / Function	Action
1	MQTT client	Publishes message
2	broker.go handler	Handler is invoked
3	<code>RejectOversizedMQTTPayload()</code>	Checks payload size
4	<code>RejectRateLimitedMQTTMessage(topic)</code>	Checks rate limit for the topic
5	<code>defaultRateLimiter.Check(topic)</code>	Runs the rate-limiter logic
6	Rate limiter	Prunes timestamps outside the time window
7	Rate limiter	Counts remaining timestamps + current message
8	<code>if count > max</code>	Logs <code>SECURITY_EVENT</code> and returns <code>true</code> , meaning reject
9	<code>else</code>	Appends timestamp and returns <code>false</code> , meaning allow
10	If rejected	Handler returns early; message is silently dropped
11	If allowed	Business logic proceeds: OCR call and Mongo write

When an HTTP request arrives on port 8080, it traverses `withRateLimit` (which prunes timestamps outside the configured window and returns HTTP 429 Too Many Requests if the per-IP threshold is exceeded), then `withCORS`, then the route mux. Per-route middleware then validates the JWT, extracts the role and pages claims, and enforces the appropriate RBAC. Only after all these layers does the request reach the controller and the repository. Repository calls that return PHI fields decrypt them in-memory just before serialization so that no plaintext PHI ever leaves the persistence boundary except through an authorized API path. The end-to-end per-request trace is:

Step	Component / Function	Action
------	----------------------	--------

1	HTTP client	Sends HTTP request
2	withRateLimit middleware	Rate-limit middleware is invoked
3	withRateLimit middleware	Extracts client IP from <code>RemoteAddr</code>
4	<code>ipLimiter.allow(ip)</code>	Checks whether the IP is allowed
5	IP limiter	Prunes timestamps outside the time window
6	<code>if len(valid) >= limit</code>	Returns <code>false</code> , meaning the request is not allowed
7	<code>else</code>	Appends current timestamp
8	IP limiter	Runs periodic cleanup if needed
9	IP limiter	Returns <code>true</code> , meaning the request is allowed
10	If not allowed	Responds with HTTP 429 Too Many Requests
11	If allowed	Passes request through <code>mux</code> , <code>CORS</code> , <code>withAuth</code> , <code>withAdminOnly</code> / <code>withAnyPage</code>
12	Handler	Final route handler is executed

Security and Compliance Relevance

The payload limit and rate limit form a layered DoS mitigation directly aligned with the project requirement. On MQTT, `message_size_limit 5242880` enforces the size cap at the broker before any application code is executed, and the Go-side `RejectOversizedMQTTPayload` re-applies the same cap with a separate environment-driven configuration so the limit cannot be silently weakened by a broker misconfiguration. The MQTT rate limiter is sliding-window and topickeyed, so a single noisy device or topic cannot exhaust the broker even if the connection itself is legitimate. On HTTP, `withRateLimit` returns 429 Too Many Requests once the per-IP threshold is exceeded and self-cleans stale IP entries every 5 minutes so the limiter cannot itself become a memory pressure source. Authentication, role-based access control, and page-scoped authorization are mandatory for every route except the explicitly public ones (`/register`, `/login`, `/broker-info`, `/uploads/`). The default admin account is bootstrapped from `ADMIN_PASSWORD` into a bcrypt hash in `server/repository/user_repository.go` `EnsureDefaultAdmin`, never persisted in plaintext, and the password field carries a `json:"-"` tag so it is excluded from API responses. Together, these controls satisfy the project requirements for protected ingestion (MQTT broker with payload and rate limits) and for secured RESTful endpoints for the frontend. The threat

model addressed by this layer is summarised below; every row corresponds to a control already in production code, not a planned mitigation:

Threat	Mitigation
MQTT flood from a compromised or misconfigured IoT device	Topic-level sliding window in <code>server/broker/rate_limit.go</code> drops excess messages before they reach OCR or MongoDB
HTTP brute-force against <code>/login</code> or <code>/register</code>	Global IP-level rate limit of 120 req/min in <code>server/routes/ratelimit.go</code> , enforced before any handler
Oversized MQTT publish, causing memory or disk DoS	5 MiB cap enforced at the broker through <code>message_size_limit</code> and re-checked in <code>server/broker/payload_limit.go</code>
Broker process resource exhaustion	Messages are rejected in-process; no goroutines, OCR jobs, or DB connections are created for dropped messages
Memory growth from tracking many client IPs	HTTP limiter sweeps stale entries every 5 minutes
Plaintext sniffing or anonymous publish	TLS-only listener with <code>require_certificate true</code> and mutual authentication
PHI exposure on the API surface	JWT authentication, role/page RBAC, and field-level AES-256-GCM decryption applied only just before JSON serialisation

```
const (
    DefaultMQTTRateLimitMessages = 60

    DefaultMQTTRateLimitWindowSeconds = 30

    mqttRateLimitMessagesEnv      = "MQTT_RATE_LIMIT_MESSAGES"
    mqttRateLimitWindowSecondsEnv = "MQTT_RATE_LIMIT_WINDOW_SECONDS"
)
```

Figure 3: MQTT Rate limits as explained above

```
// Exceeding the limit returns HTTP 429 Too Many Requests.
func withRateLimit(limit int, window time.Duration) func(http.Handler) http.Handler {
    limiter := newIPLimiter(limit, window)
    return func(next http.Handler) http.Handler {
        return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
            ip, _, err := net.SplitHostPort(r.RemoteAddr)
            if err != nil {
                ip = r.RemoteAddr
            }
            if !limiter.allow(ip) {
                http.Error(w, "Too Many Requests", http.StatusTooManyRequests)
                return
            }
            next.ServeHTTP(w, r)
        })
    }
}
```

Figure 4: Global Rate-limit deterring brute-force attacks

Tests

The HTTP rate limiter can be verified end-to-end against the running stack by sending more requests than the configured window allows and observing the 429 responses:

```
# Backend default: 120 requests/minute per IP (server/routes/init.go
RateLimitRequests/RateLimitWindow)
for i in $(seq 1 125); do
    code=$(curl -s -o /dev/null -w "%{http_code}" http://localhost:8080/broker-info)
    echo "Request $i: $code"
done
# Expected: first 120 responses are 200, the remainder become 429
```

```
for i in $(seq 1 125); do    code=$(curl -s -o /dev/null -w "%{http_code}" http://localhost:8080/broker-info);    echo "Request $i: $code"
"; done
Request 1: 200
Request 2: 200
Request 3: 200
Request 4: 200
Request 5: 200
Request 6: 200
Request 7: 200
Request 8: 200

Request 119: 200
Request 120: 200
Request 121: 429
Request 122: 429
Request 123: 429
Request 124: 429
Request 125: 429
```

```
for i in {0..6}; do python3 ./scripts/send_image.py ./medical-images/Screenshot\ 2026-01-20\ at\
13.57.50.png ; done (docker-compose.yml MQTT_RATE_LIMIT_MESSAGES=1,
MQTT_RATE_LIMIT_WINDOW_SECONDS=30)
```

```
CNP: CNP: Profesiune:SEF LUCRARI LECTOR. LocDeMunca:FACULTATEA DE AUTOMATICA SI CALCULATOARE, Bucuresti TipControl:Control medical
periodic ControlAngajare:false ControlPeriodic:true ControlAdaptare:false ControlReluare:false ControlSupraveghere:false ControlAlte:fa
lse AvizMedical:APT AvizApt:true AvizAptConditionat:false AvizInaptTemporar:true AvizInapt:true Recomandari: Data:2023-11-06 00:00:00 +
0000 UTC DataUrmExaminari:2024-11-06 00:00:00 +0000 UTC}
Processing time: 21439ms
Photo flagged for review: OCR confidence 88.7% is below the 95% threshold; extracted fields require human verification
Photo saved: photos/1779719728.jpeg (needsReview=true)
Received message on topic: register/python-sender-1
SECURITY_EVENT=MQTT_RATE_LIMIT_EXCEEDED topic=register/python-sender-1 count=2 max=1 window_seconds=30
SECURITY_EVENT=MQTT_RATE_LIMIT_EXCEEDED topic=ssproject/images/python-sender-1 count=2 max=1 window_seconds=30
Received message on topic: register/python-sender-1
SECURITY_EVENT=MQTT_RATE_LIMIT_EXCEEDED topic=register/python-sender-1 count=2 max=1 window_seconds=30
SECURITY_EVENT=MQTT_RATE_LIMIT_EXCEEDED topic=ssproject/images/python-sender-1 count=2 max=1 window_seconds=30
Received message on topic: register/python-sender-1
```

The MQTT-side guards can be exercised by tightening the limits in `docker-compose.yml`

(`MQTT_RATE_LIMIT_MESSAGES=5`, `MQTT_RATE_LIMIT_WINDOW_SECONDS=60`) and publishing more than five image messages within a minute

The authentication layer is covered by unit tests in `server/routes/user_test.go` (Register, Login, GetProfile) and by `FuzzWithAuth` in `server/routes/middleware_fuzz_test.go`, which guarantees the JWT middleware never panics on a malformed Authorization header and always returns HTTP 401 for invalid tokens.

Results

Both ingestion paths are operational, hardened, and observable. MQTT traffic is restricted to a TLS-only listener with mandatory client certificates, broker-level limits on connection count, inflight messages, queued messages and packet size, and two application-level guards (payload size and topic rate limiting) that emit structured security events when triggered. The HTTP API exposes 16 routes on port 8080 behind a per-IP rate limiter, a CORS handler, JWT authentication, role-based admin gating and page-level RBAC. The combination satisfies the project's "secure MQTT broker with payload and rate limits" requirement and the "secured RESTful APIs for the frontend" requirement in the same architecture.

Secure Data Transmission over MQTT Using mTLS

1. Purpose and Role in the Application

IoT devices publish medical images and registration events to the Mosquitto broker over MQTT. Without transport-layer protection, these messages travel as plaintext and any device on the network can connect to the broker and publish arbitrary data. mTLS addresses both problems simultaneously: it encrypts all traffic between clients and the broker, and it replaces password-based authentication with certificate-based identity



verification. Every connecting party must present a certificate signed by the shared CA - anonymous or rogue clients are refused at the TLS handshake before any MQTT data is exchanged.

2. Added, Modified, and Replaced Components

The `gen-certs.sh` script automatically generates the entire certificate chain before startup, including a 4096-bit self-signed CA valid for 10 years, a 2048-bit broker server certificate signed by the CA with SANs for broker, localhost, 127.0.0.1, and 10.0.2.2 - mobile app, and a 2048-bit client certificate for the web client. All generated private keys are secured with `chmod 600`.

The Go API uses `NewTLSConfig()` implementation in `server/main.go` to establish secure MQTT connections by loading the CA into a trusted `x509.CertPool`, loading the client certificate pair, and enforcing hostname verification with `InsecureSkipVerify: false`. On Android, `createSocketFactory()` in `MqttUploader.kt` was updated to build an `SSLSocketFactory` from certificates embedded in the APK, using `ca.crt` for trust validation and `android_client.p12` for client authentication. These resources are copied automatically into the Android project during startup.

The Mosquitto broker configuration was migrated from insecure plaintext MQTT on port 1883 with anonymous access to a TLS-only listener on port 8883. The new configuration enables CA validation, server certificates, mandatory client certificate authentication via `require_certificate true`, and identity mapping through `use_subject_as_username true`, allowing broker logs to associate MQTT connections with certificate subjects.

The Python upload utilities (`send_image.py` and `upload_folder.py`) were also migrated from plaintext TCP connections to verified TLS using `ssl.SSLContext(ssl.PROTOCOL_TLS_CLIENT)`, with hostname verification enabled, CA trust validation configured, and client certificates loaded explicitly.

Finally, `docker-compose.yml` was updated so both the broker and Go API containers mount `./secrets` as a read-only bind mount, making generated certificates available at runtime without embedding secrets directly into Docker images.

3. Execution Flow

During deployment, `start.sh` runs `gen-certs.sh` to generate a complete PKI setup: a self-signed CA, a broker certificate with SANs for local/emulator networking, and a client certificate for authenticated access. Private keys are secured with `chmod 600`. The Android client certificate is bundled into `android_client.p12`, copied with `ca.crt` into the Android app resources, and all certificates are mounted read-only into the broker and Go API containers via Docker Compose.

For each connection, clients (Go API, Python scripts, or Android app) connect to the broker on port 8883 and perform a mutual TLS handshake. The client validates the broker certificate against ca.crt and verifies hostname SANs, while the broker validates the client certificate against the same CA. After successful authentication, the broker maps the MQTT username to the certificate subject (e.g. CN=web) and establishes an encrypted TLS 1.3 session for all MQTT traffic.

Clients without a valid CA-signed certificate are rejected during the TLS handshake, before any MQTT communication occurs.

4. Security and Compliance Relevance

All MQTT communication is protected with TLS 1.3 using strong encryption (TLS_AES_256_GCM_SHA384), ensuring medical images and PHI-related payloads are never transmitted in plaintext. The system enforces mutual TLS authentication, meaning both the broker and clients must validate each other's CA-signed certificates before a connection is established. Hostname verification is also enabled in both Python and Go clients, preventing man-in-the-middle attacks by validating the broker certificate SANs.

For traceability, Mosquitto maps certificate subjects directly to MQTT usernames via `use_subject_as_username true`, allowing broker logs to identify which certificate identity published each message. Private keys are generated locally, protected with `chmod 600`, mounted read-only into containers, and excluded from Docker images and source control. Anonymous MQTT access has been completely removed by disabling port 1883 and requiring valid client certificates for all connections.

5. Tests

```
mosquitto_pub -h localhost -p 8883 -t test -m hello
```

```
mosquitto_pub -h localhost -p 8883 -t test -m hello
Error: Protocol error
```

```
sudo docker logs broker
```

```
1779719035: New connection from 172.19.0.1:51048 on port 8883.
1779719035: OpenSSL Error while trying to get the error[0]: error:0A000418:SSL routines::tlsv1 alert unknown ca
1779719035: Client 172.19.0.1 [172.19.0.1:51048] disconnected: Protocol error.
```

```
mosquitto_pub -h localhost -p 8883 -t test -m hello \
```

```
--cafile secrets/ca.crt \
```

```
--cert secrets/web.crt \
```

```
--key secrets/web.key
```

```
sudo docker logs broker
```

```

1779719132: New connection from 172.19.0.1:48238 on port 8883.
1779719132: New client connected from 172.19.0.1:48238 as auto-C27C6B0A-27B9-A28F-2D06-1087C5BFF52B (p4, c1, k60, u'CN=web,OU=WebClient,O=SS-Web,L=Bucharest,ST=Romania,C=RO').
1779719132: No will message specified.
1779719132: Client auto-C27C6B0A-27B9-A28F-2D06-1087C5BFF52B negotiated TLSv1.3 cipher TLS_AES_256_GCM_SHA384
1779719132: Sending CONNACK to auto-C27C6B0A-27B9-A28F-2D06-1087C5BFF52B (0, 0)
1779719132: Received PUBLISH from auto-C27C6B0A-27B9-A28F-2D06-1087C5BFF52B (d0, q0, r0, m0, 'test', ... (5 bytes))
1779719132: Received DISCONNECT from auto-C27C6B0A-27B9-A28F-2D06-1087C5BFF52B
1779719132: Client auto-C27C6B0A-27B9-A28F-2D06-1087C5BFF52B [172.19.0.1:48238] disconnected.

```

6. Result

All MQTT traffic between IoT devices (Android app, Python scripts), the go-api subscriber, and the Mosquitto broker is encrypted with TLSv1.3 and authenticated with mutual certificate verification. Unauthenticated clients are rejected at the TLS layer. Certificate identity is recorded in broker logs per connection. Certificates are generated per-deployment, stored outside Docker images, and mounted read-only at runtime. The Android client bundles its certificate as a PKCS12 resource compiled into the APK.

Web API Endpoints

The Web Endpoints layer is the single communication channel between the React frontend and the backend services. The API runs on TCP port 8080, exposes 19 RESTful routes, and acts as the security boundary between an untrusted browser session and the Go process, OCR sandbox, and MongoDB instance behind it.

Every request entering port 8080 traverses a fixed middleware stack — rate limiting, CORS, authentication, and authorisation — before reaching any controller logic. No route bypasses this chain.

Middleware Stack

Four middleware layers are applied in order to every incoming request:

- 1. Rate Limiting (`withRateLimit`)** An in-process per-IP sliding-window rate limiter defined in `server/routes/ratelimit.go`. It extracts the client IP from `RemoteAddr`, prunes timestamps outside the configured window, and returns HTTP 429 if the request count reaches the limit (120 requests/minute by default). A background goroutine sweeps stale IP entries every five minutes to bound memory growth.
- 2. CORS (`withCORS`)** Sets `Access-Control-Allow-*` headers on every response. OPTIONS preflight requests are short-circuited with HTTP 204 without reaching any controller.
- 3. Authentication (`withAuth`)** Applied to all protected routes. Reads the `Authorization: Bearer <token>` header, validates the JWT signature using HMAC-SHA256 against the `JWT_SECRET` environment variable, and explicitly verifies the

signing algorithm to prevent the `alg: none` attack. On success, the `email`, `role`, and `pages` claims are injected into the request context for downstream use.

4. Authorisation — two variants

- `withAdminOnly` — rejects requests where the `role` claim is not `admin` with HTTP 401. Applied to all destructive or administrative operations.
- `withAnyPage(pages...)` — checks whether the JWT's `pages` claim contains at least one of the route's required pages. Allows an operator account to be granted access to specific pages without admin privileges.

Authentication Flow

Registration (POST /register)

1. Decodes the request body into a `domain.User` struct
2. Normalises the email: `strings.ToLower + strings.TrimSpace`
3. Checks that the email is not already registered (HTTP 409 on conflict)
4. Hashes the password with `bcrypt.GenerateFromPassword` at `bcrypt.DefaultCost`
5. Saves the user to MongoDB
6. Returns HTTP 201 — no token is issued at registration

Login (POST /login)

1. Decodes and normalises email (same as registration)
2. Looks up the user by email in MongoDB
3. Calls `bcrypt.CompareHashAndPassword(storedHash, submittedPassword)` — constant-time comparison, no timing attack surface
4. On mismatch: returns HTTP 401
5. On match: builds JWT claims `{ email, role, pages, exp: now + 24h }`
6. Signs with `jwt.NewWithClaims(HS256, claims).SignedString(JWT_SECRET)`
7. Returns HTTP 200 with `{ "token": "<JWT>" }`

The token carries a 24-hour expiry. A stolen token is naturally bounded and the client clears it from `localStorage` on logout.

Authorisation Model (RBAC)

Three access tiers are enforced:

Tier	Middleware	Who qualifies
Public	None	Anyone (unauthenticated)
Page-scoped	<code>withAuth + withAnyPage</code>	Authenticated user whose <code>pages</code> claim includes the required page
Admin	<code>withAuth + withAdminOnly</code>	Authenticated user with <code>role = admin</code>

The `pages` claim is constrained server-side by a `normalizePages` helper that filters any value not in the closed set `{ photos, devices, statistics, reports, users }`. A forged or oversized claim cannot grant access to a non-existent page.

Route Table

Method	Path	Access	Notes
POST	<code>/register</code>	Public	Account registration
POST	<code>/login</code>	Public	Issues JWT token
GET	<code>/broker-info</code>	Public	MQTT host/port discovery
GET	<code>/uploads/{path}</code>	Public	Static image server
GET	<code>/profile</code>	<code>withAuth</code>	Current user profile
GET	<code>/users</code>	<code>withAuth + Admin</code>	List all users
POST	<code>/users</code>	<code>withAuth + Admin</code>	Create user
PATCH	<code>/users/{id}</code>	<code>withAuth + Admin</code>	Update user

DELETE	/users/{id}	withAuth + Admin	Delete user
GET	/photos	withAuth + Page(photos, statistics)	List photos with filters
DELETE	/photos/all	withAuth + Admin	Delete all photos
PATCH	/photos/review/{id}	withAuth + Admin	Approve/flag photo record
DELETE	/photos/{id}	withAuth + Admin	Delete single photo
GET	/devices	withAuth + Page(devices)	List devices
POST	/devices/switch	withAuth + Admin	Change device mode
POST	/devices/command	withAuth + Page(devices)	Send CAPTURE / START-LIVE / STOP-LIVE
GET	/reports/summary	withAuth + Page(reports)	Summary report
GET	/reports/expiring	withAuth + Page(reports)	Expiring checks report
GET	/reports/anonymized	withAuth + Page(reports)	PHI-free dataset
GET	/reports/performance	withAuth + Page(reports)	OCR latency and success metrics

Input Validation

Validation is applied consistently across all controllers:

- Every controller rejects unsupported HTTP methods with HTTP 405 before any logic executes
- All request bodies are decoded into typed structs — no `map[string]any` free-form parsing
- MongoDB document IDs are validated with `primitive.ObjectIDFromHex` before any database call
- Role strings are checked against a `validRole` allow-list
- Device commands are validated against the fixed set `{ CAPTURE, START-LIVE, STOP-LIVE }` before any MQTT publish — any other value returns HTTP 400

Server Configuration

The HTTP server is configured in `server/main.go` with explicit timeouts to prevent slow-loris and connection-hold attacks:

```
go
```

```
srv := &http.Server{  
    Addr:      ":8080",  
    Handler:   handler,  
    ReadTimeout: 15 * time.Second,  
    WriteTimeout: 15 * time.Second,  
    IdleTimeout: 60 * time.Second,  
}
```

Threat / Mitigation Summary

Threat	Mitigation
Credential brute-force	bcrypt hashing + per-IP rate limit (120 req/min)



JWT forgery	HS256 signature verified against <code>JWT_SECRET</code> ; signing algorithm validated explicitly
Token theft / replay	24-hour <code>exp</code> claim; token cleared on logout
<code>alg: none</code> attack	Signing method checked with <code>token.Method.(*jwt.SigningMethodHMAC)</code>
Privilege escalation via <code>pages</code> claim	Closed allow-list enforced by <code>normalizePages</code>
Password leakage in responses	<code>json:"-"</code> tag on <code>User.Password</code> ; explicit clear before <code>/profile</code> response
PHI leakage at rest	AES-256-GCM encryption in repository layer; plaintext only within the process boundary
Slow-loris / hung sockets	<code>ReadTimeout, WriteTimeout, IdleTimeout</code> on <code>http.Server</code>
Mass-delete accidents	<code>/photos/all</code> and <code>DELETE /users/{id}</code> behind <code>withAdminOnly</code>
Arbitrary device commands	Whitelist enforced before MQTT publish

```

func TestUserController_Register(t *testing.T) {
    tests := []struct {
        name      string
        inputBody  string
        mockSaveReturn error
        expectedStatus int
        expectedUser *domain.User
    }{
        {
            name:      "successful registration",
            inputBody: `{"email": "test@example.com", "password": "securepass"}`,
            mockSaveReturn: nil,
            expectedStatus: http.StatusCreated,
        },
        {
            name:      "user already exists",
            inputBody: `{"email": "test@example.com", "password": "securepass"}`,
            mockSaveReturn: errors.New("user already exists"), // Simulate existing user
            expectedStatus: http.StatusConflict,
            expectedUser: &domain.User{
                Email: "test@example.com",
            },
        },
        {
            name:      "invalid JSON",
            inputBody: `invalid-json`,
            mockSaveReturn: nil, // Save won't be called
            expectedStatus: http.StatusBadRequest,
        },
        {
            name:      "repository error",
            inputBody: `{"email": "test@example.com", "password": "securepass"}`,
            mockSaveReturn: errors.New("db error"),
            expectedStatus: http.StatusInternalServerError,
        },
    },
}

```

Figure 5: Showing authentication code snippet with checks

6. Result

The application exposes sixteen RESTful endpoints on port 8080 — four public and twelve protected — covering authentication, user management, photo lifecycle, device control, and reporting. Every request is metered by an in-process per-IP sliding-window rate limiter (120/min), framed by a CORS handler, and, on protected routes, gated by a JWT bearer authentication middleware followed by either an admin-only or a page-scoped RBAC middleware. Passwords are bcrypt-hashed with the library default cost, tokens are HMACSHA256 signed and expire in 24 hours, PHI fields are AES-256-GCM-encrypted at rest and only decrypted inside the repository immediately before serialisation, and the User.Password field has json:"- " so it can never appear in a response. The server itself runs with 15-second I/O timeouts and a 60-second idle timeout to bound socket-level abuse. Unit, integration and fuzz tests exercise every controller and every middleware.

Users Management

Permission changes apply after next login.

Create User

Pages

☐ Photos ☐ Devices ☐ Statistics ☒ Reports

Create User

Email	Password	Role	Pages	Actions
admin@test.com	*****	Admin	All	Edit Delete

Figure 6: User management page accessible by admin group

OCR Sandbox

1. Purpose and Role in the Application

The OCR Sandbox component was introduced to isolate the process of extracting text from uploaded medical images. In the initial application flow, images received by the backend needed to be processed in order to obtain readable text from medical documents. Because OCR processing involves parsing image files and using an external OCR engine, this operation was separated from the main Go API and moved into a dedicated Python service.

The main purpose of this component is to reduce the exposure of the backend API to potentially malformed or unexpected image inputs. Instead of handling OCR directly inside the main application, the Go API validates and forwards the image to an internal service named ocr-sandbox. This service is responsible only for image-based text extraction and returns a structured response containing the extracted text and OCR metadata.

This design improves the security and maintainability of the project because OCR logic is separated from the core business logic. If the OCR service fails or receives invalid data, the failure remains isolated and does not directly affect the main API process.

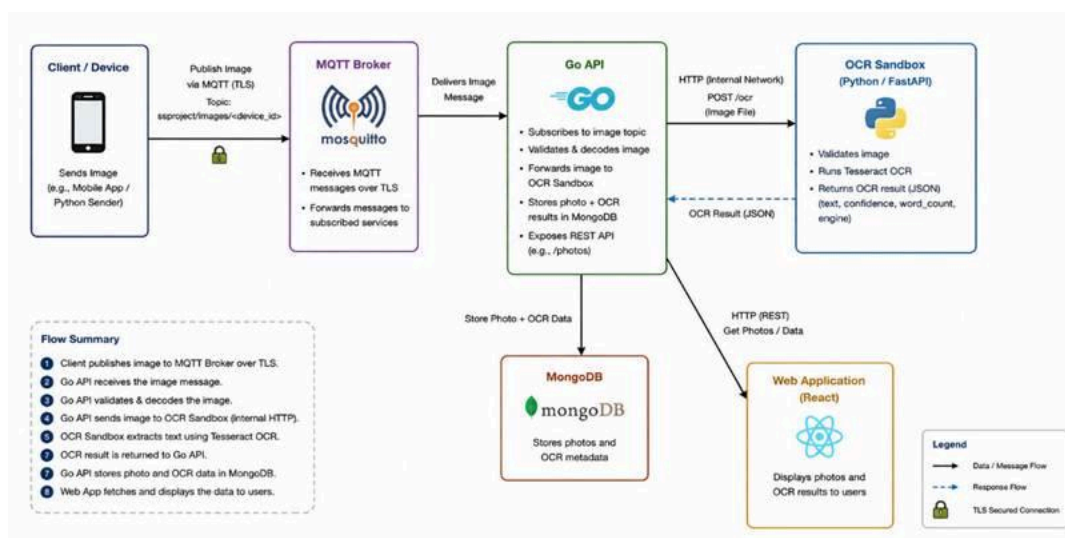


Figure 7: OCR Sandbox role in the image processing flow

2. Files and Components Added, Modified or Removed

The OCR functionality was implemented as a separate service inside the *ocr-sandbox* directory. The most important file added for this component is *ocr-sandbox/app/main.py*, which contains the *FastAPI* application and the */ocr* endpoint. This endpoint receives image files, validates them, sends them to the Tesseract OCR engine, and returns the extracted text in JSON format.

A dedicated Docker configuration was also added for the OCR service. This allows the OCR component to run as an independent container together with the rest of the application stack. The service is included in *docker-compose.yml*, where it is connected to the internal backend network and made available to the Go API through the address *http://ocr-sandbox:9000*. The OCR service returns a structured response that includes the extracted text, the average confidence score, the number of recognized words, and the OCR engine used. This makes the response easier to consume by the backend and easier to document during testing.

```
go-api:
  build: ./server
  user: "${UID}:${GID}"
  container_name: go-api
  image: stefandarius/mqtt-ss-api:latest
  env_file: .env
  environment:
    - MQTT_HOST_IP=${HOST_IP:-host.docker.internal}
    - MQTT_MAX_PAYLOAD_BYTES=5242880
    - OCR_SERVICE_URL=http://ocr-sandbox:9000
```

Figure 8: OCR Sandbox configuration in Docker Compose

The Go API was modified so that it no longer treats OCR as a direct internal responsibility. When an image is received through the MQTT broker, the backend first decodes and validates the image. After this validation step, the backend forwards the image to the OCR Sandbox through the internal Docker network. This means that the actual OCR processing is performed outside the main backend container. The Go API remains responsible for receiving the image, checking that the payload is valid, calling the OCR service, and storing the final result. The OCR Sandbox remains responsible only for extracting text from the image.

The Docker Compose configuration was also modified to include the OCR service as part of the complete application stack. The service is intentionally not exposed through a public host port. It is reachable only by other containers inside the Docker network, mainly the Go API. This prevents users or external clients from calling the OCR endpoint directly.

```

go-api      | Received message on topic: register/python-sender-1
go-api      | Received device registration: {"name": "Python Test Device", "ip": "192.168.0.131", "port": "8883"}
go-api      | Device updated: python-sender-1 (IP: 192.168.0.131, Port: 8883)
go-api      | Received message on topic: ssproject/images/python-sender-1
go-api      | Received photo from device: Python Test Device
go-api      | Image type: jpeg
ocr-sandbox | INFO: 172.18.0.5:55240 - "POST /ocr HTTP/1.1" 200 OK
go-api      | OCR confidence: 87.62%
go-api      | Extracted medical data: &{UnitateMedicala:___ Victoriei-Mihalachel 5-17 AdresaUnitateMedicala:Bd. Ion Mihalac
he, nr. 15 - 17, Sectorul 1 TelefonUnitateMedicala:+ 40 21 408 00 80, FAX: + 40 21 408 00 81 NumarFisa:1738068 SocietateUni
tate:ete.: UNIVERSITATEA NATIONALA DE STIINTA SI TEHNOLOGIE POLITEHNICA BUCURESTI. AdresaAngajator:Splaiul Independentei, n
r. 313, Sectorul 6 TelefonAngajator:Fax: Nume:Vaduva

```

Figure 9: Docker compose logs showing the Go API forward a valid image to the OCR service

3. Execution Flow

The OCR execution flow starts when a client publishes an image to the MQTT broker over TLS. The Go API receives the image message from the broker and identifies the device that sent it. Before calling the OCR service, the backend attempts to decode the image. This step acts as an initial validation layer.

If the payload is not a valid image, the Go API rejects it and does not forward it to the OCR Sandbox. This behavior was confirmed during testing with an invalid image sender script. When raw non-image bytes were published through MQTT, the Go API logged the error Failed to decode image: image: unknown format, and no new OCR request was made.

If the image is valid, the Go API sends it to the OCR Sandbox using an internal HTTP request. The OCR Sandbox validates the file, processes it with Tesseract, and returns a JSON response containing the OCR result. The Go API can then store the extracted text and related metadata together with the photo record.

```

go-api | Received message on topic: register/python-sender-1
go-api | Received device registration: {"name": "Python Test Device", "ip": "192.168.0.131", "port": "8883"}
go-api | Device updated: python-sender-1 (IP: 192.168.0.131, Port: 8883)
go-api | Received message on topic: ssproject/images/python-sender-1
go-api | Received photo from device: Python Test Device
go-api | Image type: jpeg
ocr-sandbox | INFO: 172.18.0.5:48660 - "POST /ocr HTTP/1.1" 200 OK
go-api | OCR confidence: 87.62%
go-api | Extracted medical data: &{UnitateMedicala:___ Victoriei-Mihalache1 5-17 AdresaUnitateMedicala:Bd. Ion Mihalac

```

Figure 10: Successful OCR processing logs

```

go-api | Received message on topic: register/python-invalid-sender-1
go-api | Received device registration: {"name": "Python Invalid Image Sender", "ip": "192.168.0.131", "port": "8883"}
go-api | Device registered: python-invalid-sender-1 (IP: 192.168.0.131, Port: 8883)
go-api | Received message on topic: ssproject/images/python-invalid-sender-1
go-api | Received photo from device: Python Invalid Image Sender
go-api | Failed to decode image: image: unknown format
ocr-sandbox | INFO: 127.0.0.1:54472 - "GET /health HTTP/1.1" 200 OK

```

Figure 11: Invalid image rejected before OCR processing

4. Security and Compliance Relevance

The OCR Sandbox contributes to the security of the system by isolating a potentially risky operation. OCR engines and image parsers process complex file formats, and malformed image files may cause errors or unexpected behavior. By moving this functionality into a separate container, the application limits the impact of such problems.

The service also supports the security requirement of reducing unnecessary exposure. Since the OCR Sandbox is not publicly exposed, it cannot be directly accessed from outside the Docker environment. Only the Go API can communicate with it through the internal service address. This supports the principle of least privilege and reduces the application's external attack surface.

Input validation is applied at multiple points. The Go API validates that the incoming payload can be decoded as an image before forwarding it to OCR. The OCR Sandbox also validates image input before processing it with Tesseract. This layered approach prevents invalid or malformed payloads from being processed unnecessarily.

The OCR response is also designed to avoid exposing unnecessary internal details. Instead of returning raw OCR engine errors to the client, the service uses safer error handling and returns structured OCR metadata. This helps debugging while reducing the risk of leaking sensitive implementation details.

5. Testing and Evidence

The OCR Sandbox was tested using both valid and invalid image inputs. For the valid image test, the existing sender script was used to publish an image through the MQTT broker. The broker accepted the message, the Go API received and decoded the image, and the OCR Sandbox processed it successfully. The OCR container logs showed a successful POST /ocr request with status code 200 OK.

For the invalid image test, a custom script was used to send raw non-image bytes through the same MQTT flow. The broker accepted the MQTT message, but the Go API rejected the payload during image decoding. The log message Failed to decode image:

image: unknown format confirmed that the invalid payload was stopped before reaching the OCR Sandbox.

The service health check was also verified through container logs. The OCR Sandbox responded successfully to /health requests, confirming that the service was running and available inside the Docker environment.

```
$ docker compose ps
```

NAME	IMAGE	COMMAND	SERVICE	CREATED	STATUS	PORTS
broker	eclipse-mosquitto:latest	"/docker-entrypoint..."	broker	10 hours ago	Up 10 hours	0.0.0.0:8883->8883/tcp, [::]:8883->8883/tcp
go-api	stefandarius/mqtt-ss-api:latest	"./mqtt"	go-api	10 hours ago	Up 10 hours	0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp
mongo-db	mongo:latest	"docker-entrypoint.s..."	mongo-db	10 hours ago	Up 10 hours	0.0.0.0:27019->27017/tcp, [::]:27019->27017/tcp
ocr-sandbox	ss-project-ocr-sandbox:local	"uvicorn app.main:ap..."	ocr-sandbox	10 hours ago	Up 10 hours (healthy)	9000/tcp

Figure 12: Docker containers running with OCR Sandbox enabled

6. Result

The OCR Sandbox feature was successfully integrated into the application as an isolated internal service. The main backend remains responsible for receiving and validating images, while OCR processing is delegated to a dedicated FastAPI container. This improves modularity, reduces direct exposure of the OCR engine, and provides a safer architecture for processing medical image data.

The testing results confirm that valid images are processed correctly and invalid image payloads are rejected before reaching the OCR service. Therefore, the OCR Sandbox implementation supports both the functional requirements of the application and the security objective of isolating sensitive processing operations.

Android Mobile Client - Image Capture and Offline Storage

1. Purpose and Role in the Application

The Android mobile client was implemented to provide a real mobile component for the image acquisition part of the system. Instead of relying only on the existing Python test sender script, the project now includes a mobile application that can act as a client device. Its role is to capture or select medical image data and send it into the existing secure processing pipeline.

The mobile client follows the same architectural idea as the Python sender: it communicates with the MQTT broker and publishes image messages to the image topic used by the backend. This makes the Android client part of the same system flow, where images are sent through the broker, received by the Go API, processed by the OCR Sandbox, and stored together

2. Added, Modified, and Replaced Components

The Android client was added as a separate mobile component inside the project. It was developed using Android Studio and designed to reproduce the behavior of the existing `send_image.py` script, but from a real mobile application. The client uses MQTT

communication to publish image data to the same broker topic used by the rest of the system.

The application was configured to connect to the MQTT broker using TLS. For this purpose, the mobile client uses a client certificate packaged for Android compatibility. During implementation, the certificate format had to be adapted so that the Android application could use it correctly. This included working with an Android-compatible certificate store, such as a `.p12` file, while keeping sensitive certificate material local and outside the repository.

The `.p12` file was generated from the existing client certificate and private key. The general idea was to combine the certificate, the private key, and the certificate authority information into one Android-compatible file. The command used for this type of conversion usually follows this structure:

```
openssl pkcs12 -export \  
-in web.crt \  
-inkey web.key \  
-certfile ca.crt \  
-out android_client.p12
```

During this process, OpenSSL asks for an export password. This password protects the `.p12` archive and must match the password used by the Android application when loading the certificate. The file generated from this process is then used locally by the Android client to establish a TLS-authenticated MQTT connection.

The purpose of this certificate setup is to make the Android client behave like a trusted MQTT client. When it connects to the broker, the broker can verify the client certificate and allow the mobile application to publish messages only if the certificate is valid. This improves the security of the image submission flow because the broker is not accepting anonymous or untrusted publishers.

3. Execution Flow

The mobile client participates in the image processing flow by acting as the device that initiates image submission. After the application starts, it connects to the MQTT broker using the configured broker address, port, and TLS certificate settings. Once connected, it can publish image data to the expected image topic.

The intended flow starts when the user opens the Android application and uses the camera function to capture an image. The application then prepares the image payload and publishes it to the MQTT broker. The broker forwards the message to subscribed services, including the Go API. The Go API receives the image, validates and decodes it, and then forwards it internally to the OCR Sandbox for text extraction.

The Android client can also participate in the command flow. In testing, the web interface sent a command through the broker and the broker forwarded it to the Android

listener client. This confirmed that the command topic and MQTT communication path were working.

```
broker | 1779639545: Received SUBSCRIBE from android-command-listener-1779639542881
broker | 1779639545:      ssproject/commands (QoS 1)
broker | 1779639545: android-command-listener-1779639542881 1 ssproject/commands
broker | 1779639545: Sending SUBACK to android-command-listener-1779639542881
ocr-sandbox | INFO: 127.0.0.1:47056 - "GET /health HTTP/1.1" 200 OK
broker | 1779639560: Received PUBLISH from web (d0, q0, r0, m0, 'ssproject/commands', ... (7 by
broker | 1779639560: Sending PUBLISH to android-command-listener-1779639542881 (d0, q0, r0, m0,
  'ssproject/commands', ... (7 bytes))
```

Figure 13: MQTT command received by the Android listener

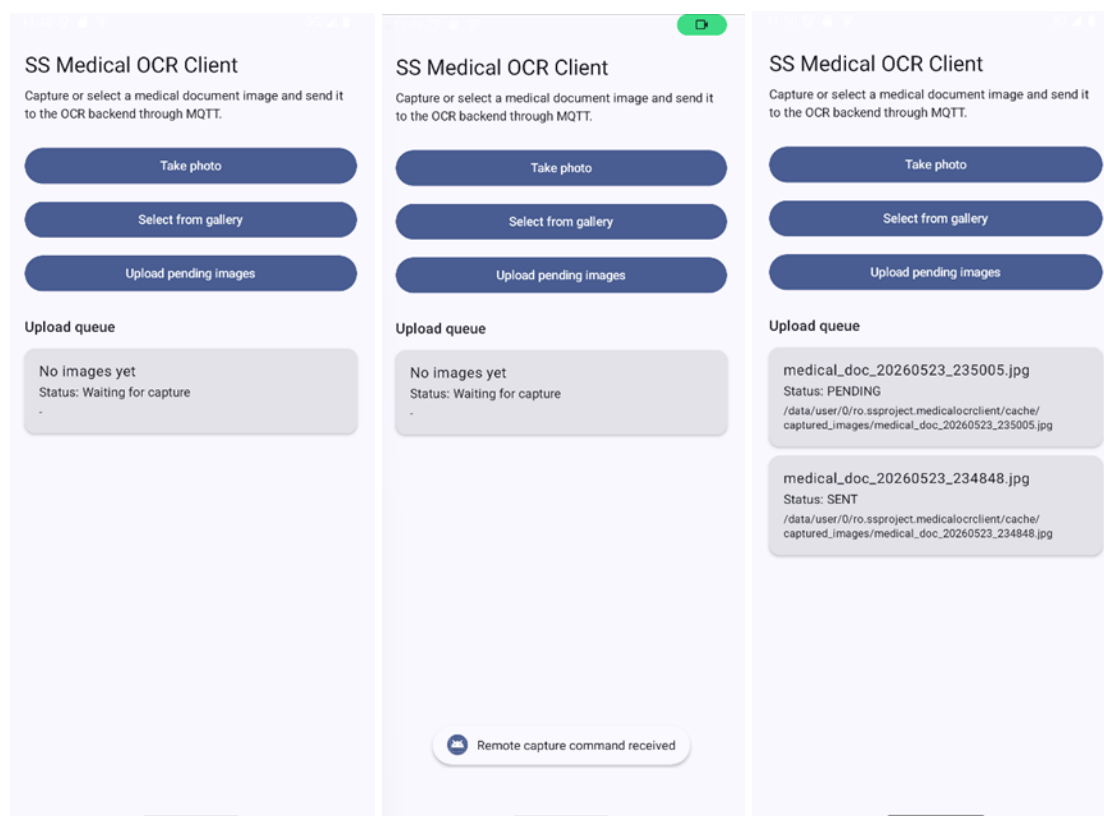


Figure 14: Android emulator running the mobile client

4. Security and Compliance Relevance

The Android client was integrated into the existing secure communication model of the project. Communication with the MQTT broker is performed over TLS, which protects the transport channel between the mobile client and the broker. This is important because the system handles medical image data, and such data should not be transmitted through an unprotected channel.

The use of client certificates also supports stronger client authentication. Instead of allowing any anonymous publisher to send images to the broker, the mobile client is configured with certificate-based access. This helps ensure that only trusted clients can participate in the image submission flow. Care was also taken not to commit sensitive local certificate files unnecessarily. The Android client certificate file is needed locally for development and testing, but private keys and certificate material should not be exposed in the repository. This is relevant for both security and good development practice.

5. Testing and Evidence

The Android client was tested mainly through the MQTT broker logs and emulator behavior. The broker logs showed that the mobile listener successfully connected to the broker and that commands sent from the web interface were forwarded to the Android command listener. This confirmed that the broker-side communication path was active and that the Android client was recognized as an MQTT participant.

```
broker | 1779637736: New connection from 172.18.0.1:45548 on port 8883.
broker | 1779637736: New client connected from 172.18.0.1:45548 as android-client-1779637734039
(p4, c1, k30, u'CN=web,OU=WebClient,O=SS-Web,L=Bucharest,ST=Romania,C=RO').
broker | 1779637736: No will message specified.
broker | 1779637736: Client android-client-1779637734039 negotiated TLSv1.3 cipher TLS_AES_256_
GCM_SHA384
broker | 1779637736: Sending CONNACK to android-client-1779637734039 (0, 0)
broker | 1779637736: Received PUBLISH from android-client-1779637734039 (d0, q1, r0, m1, 'regis
ter/android-emulator-1', ... (82 bytes))
```

Figure 15: Android client registration over secure MQTT

The following behavior was observed during testing: the broker received commands from the web interface and sent them to the Android listener client. This demonstrated that the web-to-broker-to-mobile command path was functional. Testing also involved checking TLS-related behavior. The broker logs showed TLS connections and certificate-based client identity information. This confirmed that the mobile-related communication was not plain MQTT, but part of the secured MQTT setup.

```
broker | 1779639545: Received SUBSCRIBE from android-command-listener-1779639542881
broker | 1779639545: ssproject/commands (QoS 1)
broker | 1779639545: android-command-listener-1779639542881 1 ssproject/commands
broker | 1779639545: Sending SUBACK to android-command-listener-1779639542881
ocr-sandbox | INFO: 127.0.0.1:47056 - "GET /health HTTP/1.1" 200 OK
broker | 1779639560: Received PUBLISH from web (d0, q0, r0, m0, 'ssproject/commands', ... (7 by
tes))
broker | 1779639560: Sending PUBLISH to android-command-listener-1779639542881 (d0, q0, r0, m0,
'ssproject/commands', ... (7 bytes))
```

Figure 10: MQTT command received by the Android listener

The Android client was finally tested with a complete successful image capture and publish flow. The most important final evidence should show that an image captured in the Android app reaches the broker, is received by the Go API, is processed by OCR, and

appears in the web application or `/photos` endpoint. The captured picture looks like this because it was taken from the Android emulator's virtual camera scene, not from a real medical document or text-based image. Since the image contains mostly graphical shapes and no clear readable characters, the OCR engine could not confidently recognize any words, which explains the 0% confidence result.

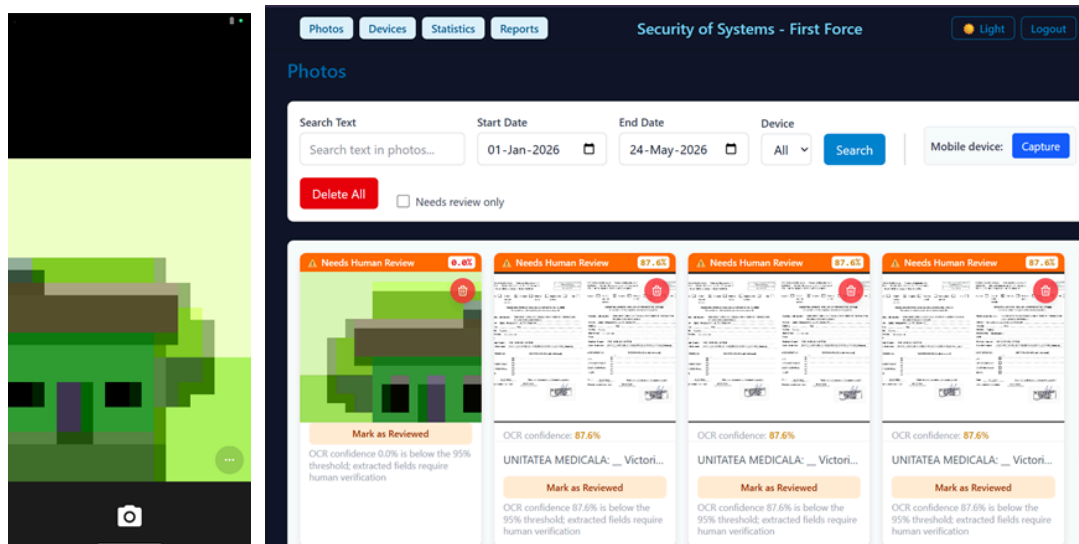


Figure 16: Successful image captured from Android client and displayed in the web application

6. Result

The Android mobile client was introduced as the real mobile-side component of the system. Its purpose is to replace a purely script-based demonstration with a more realistic client capable of interacting with the MQTT broker and submitting image data into the backend pipeline.

The implementation successfully established the mobile client as part of the MQTT architecture and confirmed that broker communication and command forwarding work. The remaining development focus is the reliable triggering of the Android camera in the emulator and the complete end-to-end image submission test from the mobile application.

Overall, the Android client strengthens the project because it demonstrates that the system is not limited to backend processing or desktop testing scripts. It provides the foundation for a realistic mobile workflow in which medical images can be captured and submitted securely through the existing MQTT, Go API, OCR Sandbox, and database pipeline.

Database Encryption (PHI Field-Level Encryption)

1. Purpose and Role in the Application

The application processes medical fitness certificates uploaded by IoT devices via MQTT. After OCR extraction, structured personal and medical data-names, CNP, employer details, medical opinions, recommendations- are persisted to MongoDB. This data constitutes Protected Health Information (PHI) under applicable data protection regulations.

Field-level encryption ensures that even if the MongoDB instance is compromised (direct database access, backup theft, misconfigured access controls), the PHI fields are stored as opaque ciphertext that cannot be read without the encryption key. The key is never stored in the database and never embedded in the application binary.

2. Added, Modified, and Replaced Components

A primary crypto utility (`server/utls/crypto.go`) implements `EncryptString` and `DecryptString`, using a key resolved from either Docker secrets (`/run/secrets/db_encryption.key`) or the `DB_ENCRYPTION_KEY` environment variable. It supports base64-encoded keys and produces ciphertext in the format `[nonce][ciphertext][tag]`. A second hardened module (`server/crypto/phi.go`) enforces stricter PHI-only encryption, requiring a 32-byte key exclusively from `PHI_ENCRYPTION_KEY` (hex) with no fallback or flexible key sizing.

A dedicated repository helper (`server/repository/photo_crypto.go`) applies encryption and decryption across 14 PHI fields in the Photo domain model, ensuring all sensitive medical and personal attributes are consistently protected at the data layer. The `gen-db-key.sh` from the script folder generates a 256-bit encryption key using OpenSSL, stores it securely in `secrets/db_encryption.key`, and is executed automatically during startup.

Repository logic was updated so encryption now occurs before database writes (Save) and decryption occurs on reads (GetPhotos, GetByID, and report queries). This shifts PHI handling from route-level logic into the repository layer, ensuring consistent data protection across all access paths. The Docker setup was updated to mount secrets as read-only into the API container.

3. Execution Flow

The system protects sensitive medical data (PHI) by encrypting it before it is stored in the database and decrypting it only when it is needed for responses.



When an IoT device uploads an image through MQTT, the system extracts relevant medical information and organizes it into a structured record. Before saving this record to MongoDB, all sensitive personal and medical fields (such as name, identification numbers, workplace, and medical details) are encrypted using strong AES-256-GCM encryption. The encryption key is securely loaded from protected system secrets or environment configuration, and each field is individually encrypted so no readable sensitive data is stored in the database.

When data is requested through the HTTP API (for example, for reports or patient records), the system reads the encrypted data from MongoDB and decrypts all sensitive fields in memory before sending the response. This ensures that the database only ever contains encrypted information, while authorized API responses return readable data.

Overall, this design ensures that sensitive medical information is always protected at rest, and only temporarily decrypted within the application when needed for legitimate access.

4. Security and Compliance Relevance

The system uses AES-256-GCM encryption to protect all sensitive medical and personal data, ensuring both confidentiality and integrity.

Encryption keys are securely managed through Docker secrets or controlled environment configuration and are never hardcoded, logged, or stored in the database. Each piece of sensitive data is encrypted with a unique randomly generated nonce, preventing pattern reuse and strengthening security.

All major personal and medical identifiers (such as names, national ID numbers, workplace details, and medical information) are encrypted at the field level before being stored in MongoDB. Only non-sensitive fields remain readable to support querying and reporting.

Because the database only contains encrypted values, compromise of MongoDB alone does not expose any meaningful personal data. Decryption is only possible within the application using the securely stored key, ensuring a strict separation between data and key material.

5. Tests


```
docker exec -it mongo-db mongosh -u admin -p changeit --authenticationDatabase admin
```

```
docker exec -it mongo-db mongosh -u admin -p changeit --authenticationDatabase admin
Current Mongosh Log ID: 6a1376e7e8bec2a8cc9df8a2
Connecting to:      mongodb://<credentials>@127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&authSource=admin&
ppName=mongosh+2.8.3
Using MongoDB:      8.2.9
Using Mongosh:      2.8.3

For mongosh info see: https://www.mongodb.com/docs/mongosh-shell/

To help improve our products, anonymous usage data is collected and sent to MongoDB periodically (https://www.mongodb.com/legal/privacy-policy).
You can opt-out by running the disableTelemetry() command.

-----
The server generated these startup warnings when booting
2026-05-24T22:06:20.811+00:00: Using the XFS filesystem is strongly recommended with the WiredTiger storage engine. See http://dochu
b.mongodb.org/core/prodnotes-filesystem
2026-05-24T22:06:21.108+00:00: Soft rlimits for open file descriptors too low
2026-05-24T22:06:21.108+00:00: For customers running the current memory allocator, we suggest changing the contents of the following
sysfsFile
2026-05-24T22:06:21.108+00:00: For customers running the current memory allocator, we suggest changing the contents of the following
sysfsFile
2026-05-24T22:06:21.108+00:00: We suggest setting the contents of sysfsFile to 0.
2026-05-24T22:06:21.108+00:00: We suggest setting swappiness to 0 or 1, as swapping can cause performance problems.
-----
```

use mqtt-streaming-server

```
test> use mqtt-streaming-server
|
switched to db mqtt-streaming-server
```

db.photos.find()

```
ocr_confidence: 88.69,
processing_time_ms: Long('20307'),
needs_review: true,
review_reason: 'OCR confidence 88.7% is below the 95% threshold; extracted fields require human verification',
unidade_medicala: 'aj+eTTOXAUFhCAIwUzEDY/URvYpduhwHMpNdI5/rmBSKLhp9UY66UURNcAzbvqYUjI4z3zPejM=',
adresa_unidade_medicala: 'qNC+hdPPkBgPPFXihL2pSHRsQJOErDxJVgStDSSRmLEdKzi9Xm9Md7J/oaqJlW/sRQLIiotaAFhN/+IneyVen07Z/eBgSdw==',
telefon_unidade_medicala: 'dig0YLC/hdg/W2xQZLkT8twvsTZyiVMCC4/QhvmZSmIrxbg4wKtdU8MahUaXRuaQQ2Ssfk9GRx6vWo75vonCyKQFET3r',
numar_fisa: '20GC06nky0MNWlbt+I5EuvfeyPtntxCKy/4q1LbafJAv1+o=',
societate_unitate: 'Fjg0L7da6bVDGhpDdZ+7lle4X99jSebFIQ3Q029r/lOtPp3//xtbph3FCysJwWp0pXXWFFD35Z7kKZ5cYCLH4ed7Q3gTQhv5aEthIv0hDAFXUS
RfmSczgv5VwSbL8xpdL2cGNFRWJD+',
adresa_angajator: '6ICShvFCF2nt6dVbu0FPp0x751hQW0+aeACP5nSn3H90cBk8vd0m9H+pHI2M2RgnkJBjI3dz23QLESFN0Ayc93nFgdxe/Q==',
telefon_angajator: '0K/xC9st1f4Jeg7LK52Ia64BkJv0FGc+qGH8+mkaHLY=',
nume: '0F41DBXqG4Ut0ZMBYaHbHlujq5aBwezpVmv4x/TUGEdYvLDPCJ4EuYrQ==',
prenume: '6pYwUlnkfeZqF/VHxXPGHcyIt0Jt0Tgqy+EPJ0kc6+IcC5VXn9RyBbBJyzqjVKQ==',
cnp: '',
profesie_functie: 'WQb7RWNFj+7KyhtlVd20z1zIbdum97J/blnkUxUT0tL3Ez0WmPHi3ak2bY591y4=',
loc_de_munca: 'YqcMIRpk7Cmt12rhUGV5G0HTHb+8yzGL/BY9tSTAVXgcZbJduN16eqBUaZUHTG5Ylv6ul8SRWp+CWds7Fg6ZW98EfrYHMDwdeqPf/vtPQ==',
tip_control: 'Control medical periodic',
control_angajare: false,
control_periodic: true,
control_adaptare: false,
control_reluare: false,
control_supraveghere: false,
control_alte: false,
aviz_medical: 'APT',
```

6. Result

All PHI fields in the photos MongoDB collection are encrypted at rest using AES-256-GCM with a per-value random nonce. Encryption and decryption are transparent to the rest of the application- the broker writes plaintext domain objects and reads them back as plaintext; the repository layer handles all crypto transitions at the persistence boundary. The encryption key is generated per-deployment, stored as a read-only host file bind-mounted into the container, and never persisted to the database or transmitted over

the network. Non-identifying fields remain queryable to support aggregations, reports, and expiration alerts without decrypting the full collection.

OCR Confidence Scoring

1. Purpose and Role in the Application

Medical fitness certificates uploaded by IoT devices are processed by Tesseract OCR. Blurry scans, skewed documents, handwriting, or low-quality captures degrade text extraction accuracy. Blindly trusting OCR output on PHI (names, CNP, medical opinions) risks silently storing incorrect data with no indication that human review is needed.

The OCR confidence score measures Tesseract's word-level certainty for every processed image and drives an automated triage decision: images where extraction is reliable are stored directly; images below the threshold are flagged and routed to a human reviewer.

2. Added, Modified, and Replaced Components

An `OCRConfidence float64` field was added to the Photo domain model and persisted to MongoDB. Two review-state fields (`NeedsReview bool`, `ReviewReason string`) are written at save time, and two audit fields (`ReviewedBy string`, `ReviewedAt *time.Time`) are written when an admin approves a flagged record via `PATCH /photos/review/{id}`.

The OCR sandbox returns confidence, `recognized_word_count`, and engine alongside the extracted text. Confidence is computed from Tesseract's word-level array using `image_to_data`, excluding non-word regions (`confidence < 0`).

Broker logic evaluates three flagging conditions in sequence after OCR returns: extraction error, confidence below 95%, or medical parser failure. All three can combine into a single semicolon-separated `ReviewReason`. The admin review endpoint writes `reviewed_by`, `reviewed_at`, and clears `needs_review`. Performance metrics queries expose flagged count, auto-processed count, and average confidence through the reporting API.

The photo card UI was extended to show the confidence score with colour coding (green $\geq 95\%$, orange $\geq 75\%$, red $< 75\%$), a needs-review banner with reason text, and a one-click approval button. The statistics page surfaces the count of photos with `needs_review: true`.

3. Execution Flow

When an IoT device publishes an image, the broker calls the OCR sandbox, which runs Tesseract and returns text alongside a word-level confidence average. The broker evaluates the result: if extraction failed, confidence is below 95%, or the medical parser could not extract all fields, `NeedsReview` is set to true with a descriptive reason. The photo is then saved to MongoDB with the confidence score and review flag attached.

When an admin opens a flagged photo, they see the confidence score, the flagging reason, and a review button. Clicking it calls PATCH /photos/review/{id}, which clears the flag and writes the reviewer's email and a UTC timestamp as an audit trail.

4. Security and Compliance Relevance

Photos below 95% confidence are flagged before any PHI field is treated as authoritative, preventing incorrect data from entering the system silently. Both extraction errors and low-confidence results produce an explicit ReviewReason, so no failure is stored without an explanation. The review backlog is visible to administrators through the statistics dashboard and report summary.

5. Tests

python3 ./scripts/send_image.py ./medical-images/Screenshot\ 2026-01-20\ at\ 13.57.50.png

```
python3 ./scripts/send_image.py ./medical-images/Screenshot\ 2026-01-20\ at\ 13.57.50.png
Device ID: python-sender-1
Connecting to 127.0.0.1:8883...
Connected to MQTT Broker!
Registering device: register/python-sender-1
Publishing image to: ssproject/images/python-sender-1
Sending file: ./medical-images/Screenshot 2026-01-20 at 13.57.50.png
Message published successfully!
✓ Device 'python-sender-1' registered and photo sent!
Topic: ssproject/images/python-sender-1
```

```
Received message on topic: ssproject/images/python-sender-1
Received photo from device: Python Test Device
Image type: jpeg
OCR confidence: 88.69%
Extracted medical data: &{UnitateMedical: __ Victoriei-Mihalache 1 5-17 AdresaUnitateMedical: Bd. Ion Mihalache, nr. 15 - 17, Sectorul 1
TelefonUnitateMedical: + 40 21 408 00 80, FAX: + 40 21 408 00 81 NumarFisa: 1738068 SocietateUnitate: etc.: UNIVERSITATEA NATIONALA DE S
TIINTA SI TEHNOLOGIE POLITEHNICA BUCURESTII. AdresaAngajator: Splaiul Independentei, nr. 313, Sectorul 6 TelefonAngajator: Fax: Nume: Vaduv
a
PRENUME Prenume: Jan alexandru
CNP CNP: ProfesiaFuncie: SEF LUCRARI LECTOR. LocDeMunca: FACULTATEA DE AUTOMATICA SI CALCULATOARE, Bucuresti TipControl: Control medical
periodic ControlAngajare: false ControlPeriodic: true ControlAdaptare: false ControlReluare: false ControlSupraveghere: false ControlAlte: fa
lse AvizMedical: APT AvizApt: true AvizAptConditionat: false AvizInaptTemporar: true AvizInapt: true Recomandari: Data: 2023-11-06 00:00:00 +
0000 UTC DataUrmExaminari: 2024-11-06 00:00:00 +0000 UTC
Processing time: 21470ms
Photo flagged for review: OCR confidence 88.7% is below the 95% threshold; extracted fields require human verification
Photo saved: photos/1779720367.jpeg (needsReview=true)
```

Dark Mode Toggle

The platform implements a system-wide dark mode toggle, allowing users to switch between light and dark themes. The preference is persisted across sessions via `localStorage`, so the selected theme is restored automatically on the next visit.

Implementation

ThemeContext (`client/src/contexts/ThemeContext.tsx`)

A React context manages the theme state globally. On initialisation, it reads the stored preference from `localStorage`. When the user toggles the theme, it updates the `dark` class on the root `<html>` element and writes the new preference to `localStorage`.

Tailwind v4 Configuration (`client/src/index.css`)

Tailwind v4 uses CSS-based configuration. The `@custom-variant` directive registers a `dark` variant that activates when any ancestor element carries the `dark` class:

This means any component can use `dark`-prefixed utility classes without additional configuration.

Toggle Button (`client/src/App.tsx`)

The toggle is exposed as a button in the navbar, available on every page regardless of authentication state. The label updates dynamically to reflect the current mode:

Body background (`client/src/index.css`)

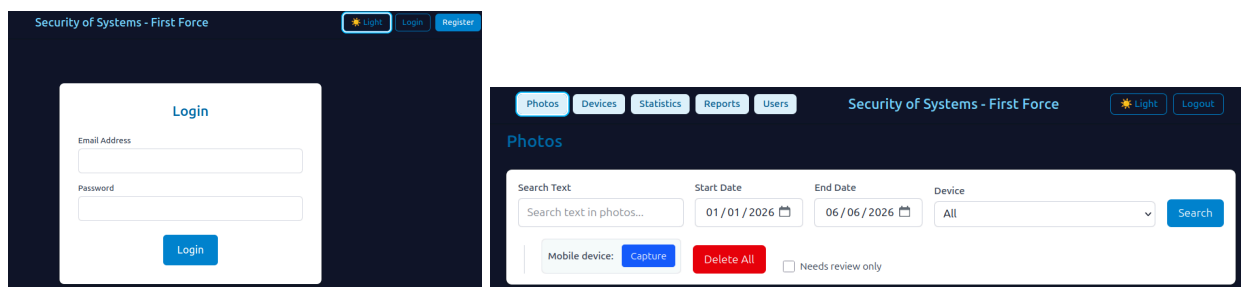
The page background transitions smoothly between themes:

Coverage

Dark mode `dark:` variants are applied on all major UI surfaces:

- **Navbar** — background (`dark:bg-gray-900`) and title text (`dark:text-sky-300`)
- **Page backgrounds** — all layout containers use `dark:bg-gray-900` or `dark:bg-gray-800`
- **White panels** — cards and tables with `bg-white` have `text-gray-900` added explicitly to prevent text from inheriting the dark theme colour and becoming invisible against the light panel background

The last point was addressed as a tracked bug fix (GitHub Issue) after a teammate added new report pages — the fix added `text-gray-900` to all white panel containers in the Reports page, and the issue was closed automatically via the commit message (`Closes #28`).



Security and Compliance

CI/CD Evidence Plane Workflow

The project includes a dedicated GitHub Actions workflow named **Evidence Plane** - **SBOM** and **Audit Artifacts**, implemented in `.github/workflows/evidence-plane.yml`. This workflow was added to create an automated evidence layer for the project. Its purpose is to collect and store artifacts that demonstrate the state of the application from a testing, dependency, and security perspective.

The workflow creates structured evidence directories for different categories of outputs. These include software bill of materials files, test outputs, security audit reports, and logs. This organization makes the generated results easier to inspect and include in the final documentation. The workflow also supports artifact upload. This means that after a workflow run finishes, the generated evidence can be downloaded from the GitHub Actions run page. The reports contain the outputs produced by the workflow, including dependency information, test execution results, security scan outputs, and logs. These files are used as proof that the project includes a repeatable CI/CD evidence generation process.

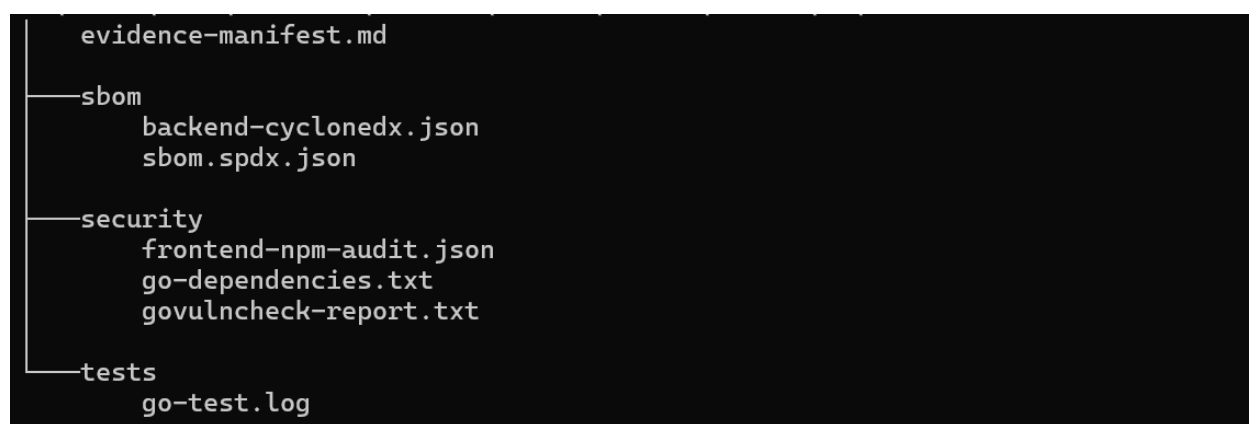


Figure 17: Evidence directories and files generated by the workflow

The Evidence Plane workflow is not a feature used directly by the end user. Instead, it is a development and compliance feature. It helps prove that the project was tested, that dependencies were checked, and that security-related artifacts were collected in a repeatable way.

The workflow runs automatically on pull requests and on pushes to the **main** branch. It can also be started manually using the **workflow_dispatch** option. This makes

it useful both during development and during final project evaluation, because the team can generate updated evidence whenever needed.

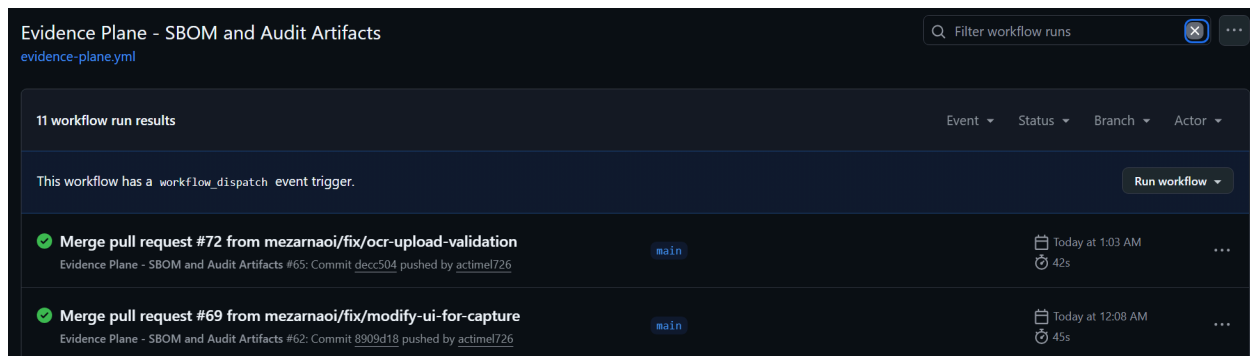


Figure 18: Evidence Plane workflow execution in GitHub Actions

CI/CD Pipeline Architecture

The project implements a **segregated multi-pipeline CI/CD architecture** on GitHub Actions, designed around the principle that different pipeline stages have different execution triggers, time budgets, and responsibilities. Rather than a single monolithic pipeline, five specialized workflows run independently to avoid blocking fast feedback with slow processes (e.g. fuzzing does not block PR merges).

All changes to the `main` branch are gated — direct pushes are blocked by branch protection rules. Every change must arrive via a Pull Request and pass the required CI checks before merge is permitted. This is enforced at the repository level and cannot be bypassed.

Pipeline 1 — CI Gateway (`ci.yml`)

Trigger: Every Pull Request targeting `main` and every push to `main`

Purpose: Fast security and quality gate that blocks merge on any new finding. This is the mandatory checkpoint referenced in the course requirements — no change integrates without passing this pipeline.

Jobs (run in parallel):

ESLint (Frontend)

- Installs Node 20 and project dependencies via `yarn install --frozen-lockfile`
- Runs `yarn lint` against the TypeScript/React codebase using the project's `eslint.config.js`

- Fails the PR on any error-level finding (warnings are reported but do not block)
- Covers: unused variables, unsafe patterns, React hook violations, import issues

go vet + gosec (Backend)

- Installs Go 1.23 and system-level Tesseract/Leptonica dependencies required by the OCR library
- Runs `go vet ./...` to catch compiler-level issues: unreachable code, incorrect format strings, suspicious constructs
- Installs and runs `gosec` — a Go-specific security scanner covering 17 CWE categories including SSRF (G704), path traversal (G703), HTTP server without timeouts (G114), unhandled errors (G104), insecure file permissions (G301/G306), and secrets exposed in serialized structs (G117)
- All 17 findings from the initial scan were resolved: critical ones were patched (HTTP timeouts added, file permissions tightened, password field excluded from JSON serialization), and accepted risks were documented with `#nosec` annotations including justification comments

Artifacts generated: None (gate only — pass/fail result)

Pipeline 2 — Unit Tests & Coverage (`tests.yml`)

Trigger: Every Pull Request targeting `main` and every push to `main`

Purpose: Validates functional correctness of all Go handlers and enforces a minimum code coverage threshold to prevent coverage regression.

Execution:

- Runs `go test -v -race -coverprofile=coverage.out -covermode=atomic ./routes/... ./broker/...`
- The `-race` flag enables Go's race detector, catching concurrent memory access bugs that would not surface in single-threaded test runs
- Coverage is computed with `go tool cover -func=coverage.out` and the total is extracted and compared against the threshold
- Build fails if total coverage drops below **20%**, preventing regressions as the codebase grows

Test suite (729 lines across 4 files):

- `routes/user_test.go` — Register, Login, GetProfile handlers; includes fuzz seed cases

- `routes/photo_test.go` — GetPhotos with filters, method validation, timestamp parsing
- `routes/device_test.go` — GetDevices with RBAC enforcement, SwitchDeviceMode, method guards
- `broker/broker_test.go` — MQTT device registration handler

Notable test design decisions:

- Mock objects generated via `go.uber.org/mock/gomock` for all repository interfaces, ensuring handlers are tested in isolation without a live database
- The `MockPhotoRepository` was extended with `GetByID`, `Delete`, and `DeleteAll` methods to fully satisfy the `PhotoRepository` interface after the interface was updated
- The Login test generates a real bcrypt hash at runtime (`bcrypt.GenerateFromPassword`) rather than using a hardcoded hash string, ensuring the test remains valid across bcrypt library updates
- Device controller tests verify RBAC enforcement: requests with role `user` must receive HTTP 401 before any repository method is called

Artifacts generated: `coverage.out` uploaded as a GitHub Actions artifact, retained for 30 days, downloadable for local analysis with `go tool cover -html=coverage.out`

Pipeline 3 — Static Analysis (`analysis.yml`)

Trigger: Every Pull Request, every push to `main`, and weekly on Monday at 02:00 UTC (scheduled)

Purpose: Deep code quality and security analysis beyond what `go vet` and `gosec` provide. Results are exported in SARIF format and integrated into GitHub's Security → Code Scanning dashboard, where findings are tracked, assigned, and resolved.

Jobs (run in parallel):

`golangci-lint`

- Installs `golangci-lint` at latest version (v2.x) directly from the Go toolchain
- Configured via `.golangci.yml` with 11 enabled linters:
 - `errcheck` — ensures all returned errors are handled
 - `gosimple` / `staticcheck` — detects simplifiable code and suspicious patterns
 - `govet` — mirrors `go vet` for consistency
 - `ineffassign` — finds assignments whose value is never used

- **unused** — identifies exported identifiers never referenced outside the package
- **gosec** — security-focused linting (overlaps with Pipeline 1 for defence-in-depth)
- **gocritic** — diagnostic and security tag checks
- **gocyclo** — flags functions with cyclomatic complexity above 15
- **misspell** — catches common English spelling errors in comments and strings
- **bodyclose** — verifies HTTP response bodies are closed to prevent connection leaks
- **noctx** — detects HTTP requests created without a context
- Output is written in SARIF format to **golangci-results.sarif**; a valid empty SARIF is generated as fallback if linting produces no output
- Results uploaded to GitHub Security tab via **github/codeql-action/upload-sarif@v4**

CodeQL

- GitHub's semantic code analysis engine, initialised for both **go** and **javascript-typescript**
- Uses the **security-and-quality** query suite — a superset of the default queries, covering injection, authentication bypass, data flow vulnerabilities, and code quality patterns
- Autobuild compiles the Go server with all dependencies (including Tesseract) to enable data-flow analysis across the full call graph
- Results uploaded to GitHub Security tab under a separate **codeql** category
- Runs on a schedule in addition to PRs to catch vulnerabilities introduced by dependency updates

Artifacts generated: SARIF files visible in GitHub Security → Code Scanning; findings are retained indefinitely and tracked across commits

Pipeline 4 — Fuzzing (**fuzzing.yml**)

Trigger: Scheduled weekly (Saturday at 22:00 UTC) and manually via **workflow_dispatch** with configurable **fuzz_time** parameter (default: 5 hours per target; accepts values like **2m** for quick validation or **5h** for production runs)

Purpose: Property-based security testing. Unlike unit tests that verify known inputs produce known outputs, fuzzing generates millions of random and mutated inputs to find inputs that cause panics, excessive resource consumption, or unexpected behaviour. This is directly relevant to the medical platform's threat model: the MQTT broker accepts image

payloads from any connected device, and the OCR engine processes those images — both are attack surfaces for malformed or adversarial input.

Fuzz targets (run sequentially, each for the configured duration):

FuzzUserController_Register (pre-existing in codebase)

- Seeds: valid JSON, empty fields, short values, invalid JSON, truncated JSON
- Tests: the register handler never panics regardless of request body content
- Found during initial run: correctly handles all malformed inputs with appropriate HTTP error codes

FuzzUserController_Login

- Seeds: valid credentials, empty email, missing fields, very long email strings
- Mock returns a user with a runtime-generated bcrypt hash for valid email inputs; returns `not found` error for unrecognised emails
- Tests: the login handler never panics; bcrypt comparison is safe against arbitrary password strings

FuzzPhotoController_GetPhotos

- Seeds: empty query params, valid Unix timestamps, invalid strings, very large numbers
- Query parameters are URL-encoded via `url.Values` before constructing the request URL, preventing `httptest.NewRequest` panics on characters that would make the URL syntactically invalid — this bug was discovered and fixed during the first fuzzing run
- Tests: the GetPhotos handler correctly rejects invalid timestamps with HTTP 400 and never panics on any query string

FuzzWithAuth (internal package test)

- Seeds: empty Authorization header, `Bearer` with no token, a valid-format but invalid JWT, a 500-character random string
- Tests: the `withAuth` middleware never panics and always returns HTTP 401 for any invalid or malformed token

FuzzImageDecode

- Seeds: JPEG magic bytes, PNG magic bytes, plain text, null bytes, repeated 0xFF bytes
- Calls `image.DecodeConfig` directly with arbitrary byte slices

- Tests: Go's image decoder never panics on any byte sequence; errors are returned, not propagated as panics

FuzzOCRProcessing

- Seeds: JPEG header bytes, non-image data, empty slice, repeated 0xAB bytes
- Creates a live `gosseract.Client` and calls `SetImageFromBytes` followed by `Text()` with arbitrary data
- Tests: Tesseract's Go bindings never panic when fed arbitrary bytes — directly validates the sandboxing threat model

Corpus management:

- Fuzzing corpus (interesting inputs discovered during runs) is cached between executions using GitHub Actions cache with a SHA-keyed entry, ensuring each run benefits from previously discovered edge cases
- Corpus is also uploaded as an artifact (retained 90 days) for offline analysis

Artifacts generated: Fuzz corpus cache, `fuzz-corpus-{run_id}` artifact containing all seed files and discovered edge cases

Pipeline 5 — Release (`release.yml`)

Trigger: Push of a version tag matching `v*.*.*` (e.g. `git tag v1.0.0 && git push origin v1.0.0`)

Purpose: Produces a reproducible, versioned release artifact that can be deployed by anyone without access to the source code. Implements the SCM & Repositories and Build/Release requirements.

Steps:

1. Authenticates to GitHub Container Registry (ghcr.io) using the workflow's `GITHUB_TOKEN`
2. Builds the Go server Docker image from `./server/Dockerfile` using `docker/build-push-action@v6`
3. Pushes two tags: the specific version (`ghcr.io/mezarnaoi/ss-project:v1.0.0`) and `latest`
4. Prepares a release ZIP containing: `docker-compose.yml`, `.env.example` (with safe placeholder values), and `sbom.cdx.json`
5. Creates a GitHub Release with a human-readable description, download links for the ZIP, and the SBOM file attached directly



Versioning strategy: Semantic versioning (**MAJOR.MINOR.PATCH**). The tag is extracted from **GITHUB_REF** and propagated to both the Docker image tag and the release name, ensuring the running container version is always traceable back to the exact source commit.

To deploy a release:

bash

```
docker pull ghcr.io/mezarnaoui/ss-project:v1.0.0
```

```
# Download release ZIP, configure .env.example → .env
```

```
docker compose up -d
```

Artifacts generated: Docker image on ghcr.io, GitHub Release page, versioned ZIP with deployment files, SBOM attached to release

AI Agent — Automated PR Security Review

Trigger: Every Pull Request opened or updated against **main**

Purpose: Implements the AI-assisted governance layer described in the course architecture. Every code change receives an automated security review from a large language model before human review, providing a first-pass analysis that flags potential PHI exposure, authentication issues, and error handling gaps. This demonstrates the human-in-the-loop principle: the AI proposes observations, but a human engineer makes the final merge decision.

Implementation:

- A Python script (**.github/scripts/ai_review.py**) fetches the PR diff via the GitHub REST API, filtering to **.go**, **.ts**, **.tsx**, and **.py** files and capping at 500 changed lines to stay within model context limits
- The diff is sent to **GitHub Models API** (Azure AI Inference endpoint) using the workflow's built-in **GITHUB_TOKEN** — no external API key required, no IP blocking issues
- Model: **meta-llama-3.1-8b-instruct**, prompted as a security reviewer specialised in medical PHI platforms
- The review prompt specifically requests analysis of: security vulnerabilities, PHI data handling, error handling gaps, and code quality issues with file and line number references

- The model's response is posted as a comment on the PR with a clear disclaimer that the review is AI-generated and human approval is still required
- Retry logic with exponential backoff handles transient rate limiting (up to 5 attempts, 10–50 second intervals)

Example output: The agent reviews changes to authentication middleware and flags if bearer token validation could be bypassed, or if a new endpoint omits the `withAuth` wrapper, before any human reviewer sees the PR.

Artifacts generated: PR comment visible to all reviewers; no persistent artifact

Branch Protection & Merge Policy

The `main` branch has the following protection rules enforced at the repository level:

- Direct pushes to `main` are blocked — all changes require a Pull Request
- The following status checks must pass before merge is permitted: `ESLint (Frontend)`, `go vet + gosec (Backend)`, `Unit Tests & Coverage`
- The `analysis.yml` and `fuzzing.yml` pipelines do not block merge (their findings are tracked in the Security tab and addressed in subsequent PRs)
- Auto-merge is available: once all required checks pass, a PR can be configured to merge automatically without manual intervention

SARIF Integration & Security Dashboard

Both `golangci-lint` and `CodeQL` export results in SARIF (Static Analysis Results Interchange Format), which is uploaded to GitHub's Code Scanning API. This populates the **Security** → **Code Scanning** dashboard with:

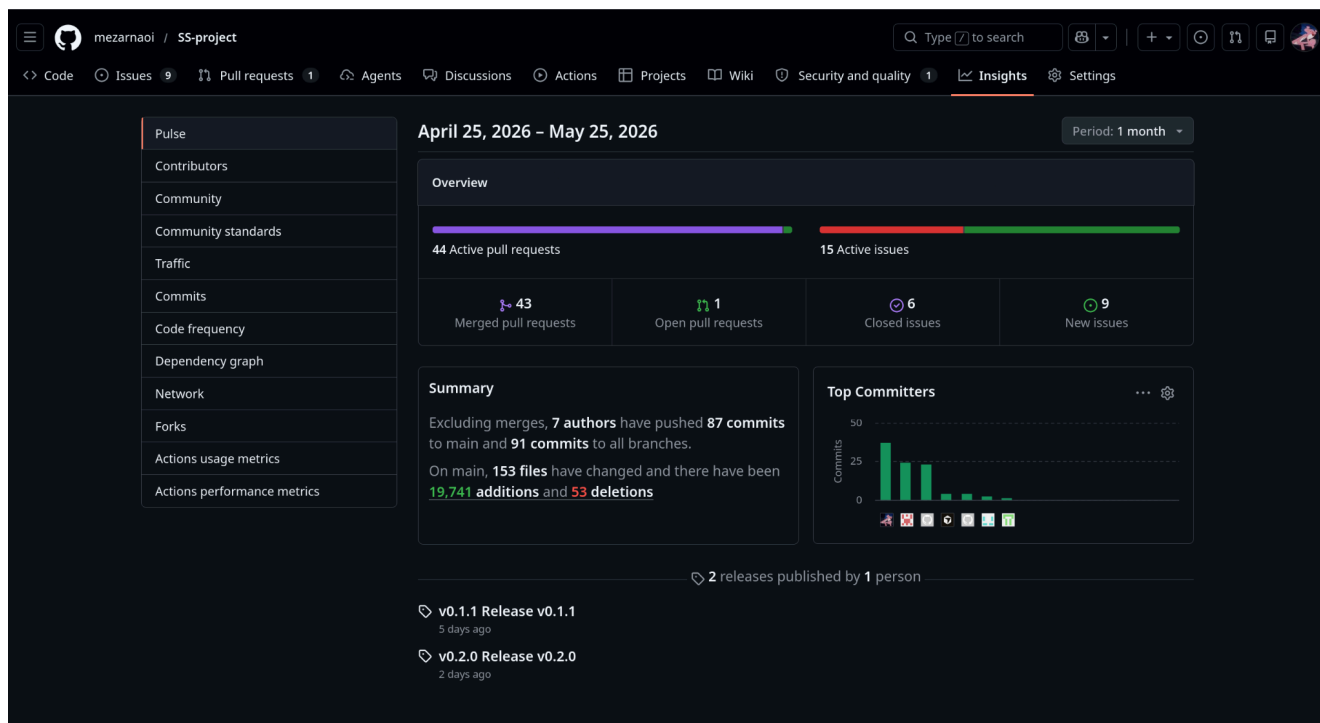
- Finding title, severity, CWE category, file path, and line number
- Historical tracking (open, dismissed, fixed states)
- Automatic deduplication across runs
- Differentiation by tool (`golangci-lint` findings vs `CodeQL` findings shown separately)

This fulfils the course requirement for "results exported in SARIF format integrated into a centralised control panel."

Pipeline Summary Table

Pipeline	Trigger	Duration	Blocks Merge	Key Output
<code>ci.yml</code>	Every PR / push to main	~3 min	Yes	ESLint + gosec pass/fail
<code>tests.yml</code>	Every PR / push to main	~4 min	Yes	Coverage report artifact
<code>analysis.yml</code>	Every PR + weekly	~8 min	No	SARIF → Security dashboard
<code>fuzzing.yml</code>	Weekly + manual	Up to 30h	No	Fuzz corpus artifact
<code>release.yml</code>	Version tag push	~5 min	N/A	Docker image + GitHub Release
<code>ai-review.yml</code>	Every PR	~1 min	No	PR comment with security review

Team Contributions



Top Committers			
Category	Commits		
mezarnai	37		
sorin0popa	24		
Deni	23		
cursoragent	4		
Victor	4		
ripsterk (tot Victor)	2		
actimel726 (tot Deni)	1		
		Download CSV	

Medical Reports

The platform implements a dynamic reporting system accessible via the **Reports** page (4 tabs) and the **Statistics** page. All reports query live data from MongoDB and are protected by JWT authentication — only authenticated users can access report endpoints.

Report 1 — Medical Opinion Summary (Fitness Breakdown)

Location: Reports → Summary tab → Medical Opinion Breakdown panel

What it shows: The total count of employees categorised by their medical fitness verdict: Fit (APT), Conditionally Fit (APT Conditionat), Temporarily Unfit (Inapt Temporar), and Unfit (Inapt). Each category is displayed as a card with a distinct colour code for immediate visual identification.

Example from live data: 30 Fit · 10 Conditionally Fit · 9 Temporarily Unfit · 2 Unfit

Medical relevance: HR and occupational health teams can instantly see the overall fitness distribution across all employees without browsing individual records. The breakdown helps identify whether the workforce health profile is within acceptable limits and flags categories that require managerial action (e.g. Unfit employees who cannot legally perform their role).

Implementation: The backend `/reports/summary` endpoint runs a MongoDB aggregation grouping documents by `aviz_medical` field. The frontend renders the result as colour-coded summary cards.

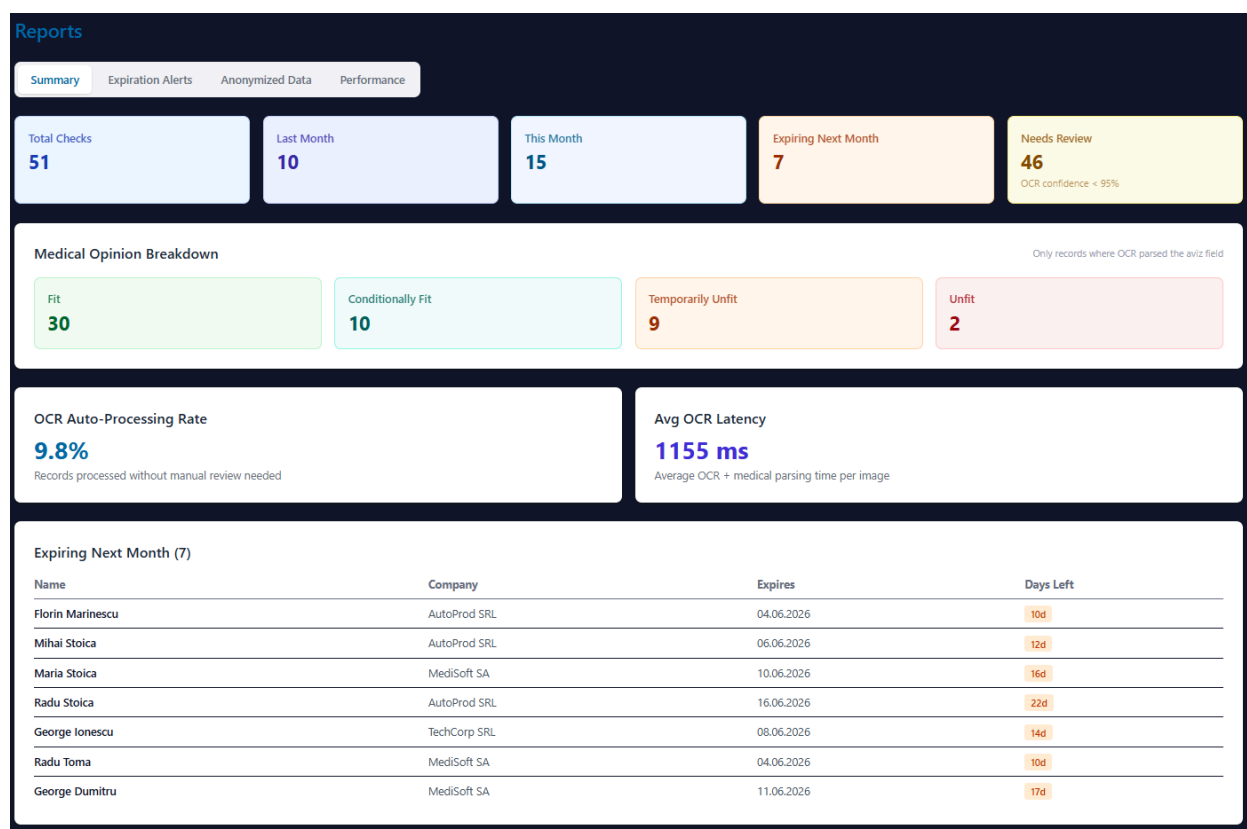


Figure 19: Reports dashboard page

Report 2 — Expiration Alerts (Who Expires Next Month)

Location: Reports → Expiration Alerts tab

What it shows: A table of all employees whose occupational medicine check (`data_urm_examinari`) expires within a configurable window (default: next 30 days). Rows are colour-coded by urgency: red for ≤ 7 days (Critical), orange for ≤ 30 days (Soon), and yellow for > 30 days (Upcoming). Columns include Employee, Company/Unit, Workplace, Expiry Date, Days Left, and Medical Opinion.

Example from live data: George Florea — BuildMax SA — expires 27.05.2026 — 2 days remaining (Critical, APT CONDITIONAT)

Medical relevance: This is the most operationally critical report. Romanian labour law requires periodic occupational medicine checks (*medicina muncii*). An expired check means the employee cannot legally continue working in their role. This report allows HR to proactively schedule renewals rather than react to expired documents.

Implementation: The backend queries all documents where `data_urm_examinari` is between `now` and `now + N days`. The frontend renders the "Days Left" badge with conditional colour logic based on the numeric value.

Employee	Company / Unit	Workplace	Expires	Days Left	Aviz
George Florea	BuildMax SA	BuildMax SA	27.05.2026	2 days	APT CONDITIONAT
Florin Marinescu	AutoProd SRL	AutoProd SRL	04.06.2026	10 days	APT
Radu Toma	MediSoft SA	MediSoft SA	04.06.2026	10 days	APT
Mihai Stoica	AutoProd SRL	AutoProd SRL	06.06.2026	12 days	APT CONDITIONAT
George Ionescu	TechCorp SRL	TechCorp SRL	08.06.2026	14 days	APT
Maria Stoica	MediSoft SA	MediSoft SA	10.06.2026	16 days	APT
George Dumitru	MediSoft SA	MediSoft SA	11.06.2026	17 days	APT
Radu Stoica	AutoProd SRL	AutoProd SRL	16.06.2026	22 days	APT

Figure 20: Showcase of reports that will meet expiry soon

Report 3 — Anonymized Dataset for Research

Location: Reports → Anonymized Data tab

What it shows: Two sub-sections:

- **Fitness by Profession** — cards showing Fit vs Other count for each job category (Sofer, Manager, Contabil, Medic, Mecanic, etc.) without any personal identifiers
- **Anonymized Record Table** — full dataset with Control Type, Medical Opinion, Company, Role, Date, Next Exam, and OCR Confidence — but with Name, CNP, and personal identifiers removed

Example from live data: Sofer: 4 Fit · 1 Other; Mecanic: 2 Fit · 1 Other

Medical relevance: Occupational health research and compliance reporting often require aggregate data but are legally prohibited from including PHI (Patient Health Information). This report allows anonymized data export compliant with GDPR and medical data protection regulations. Researchers can analyse fitness trends by profession or company without accessing personal records.

Implementation: The API endpoint strips **nume**, **prenume**, and **cnp** fields before returning data. The frontend renders the anonymized records in a table and computes profession-level aggregates client-side.

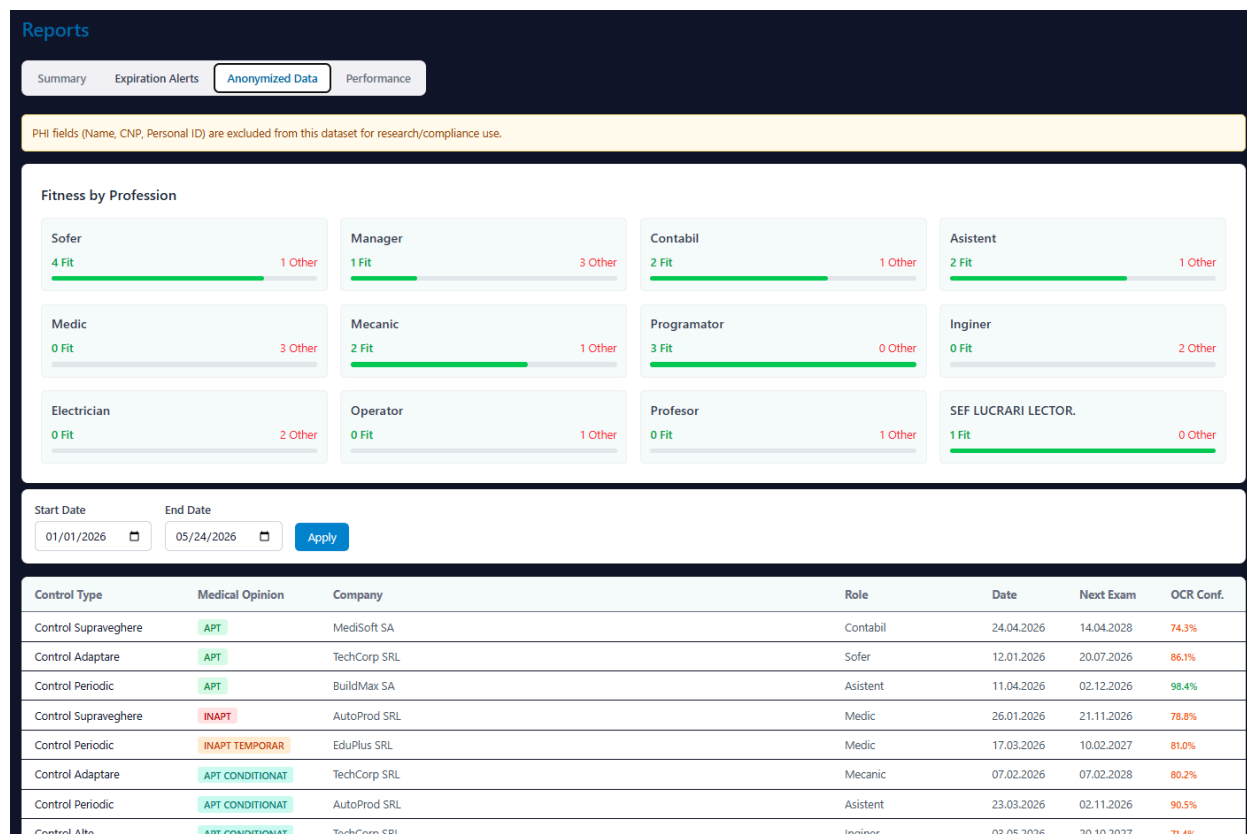


Figure 21: Anonymized Data Dashboard with different metrics

Report 4 — System Performance Metrics

Location: Reports → Performance tab

What it shows:

- **Auto-processing success rate** — percentage of records processed without requiring manual review (OCR confidence $\geq 95\%$)
- **Average OCR confidence** — mean confidence score across all processed images
- **Manual reviews completed** — ratio of reviewed vs total records needing review
- **Processing Latency** — Average, Minimum, and Maximum time from image receipt to OCR + medical field extraction completion

Example from live data: Auto-processing rate: 9.8% · Avg confidence: 85.6% · Avg latency: 1155 ms · Min: 220 ms · Max: 8610 ms

Medical relevance: System administrators and medical unit managers need visibility into OCR accuracy to trust extracted data. A low auto-processing rate (many records below 95% confidence) signals image quality issues or document format mismatches requiring operator intervention. Latency metrics help size infrastructure for high-volume periods (e.g. annual periodic checks).

Implementation: The backend computes confidence statistics via MongoDB aggregation on the `ocr_confidence` field and latency from the `processing_latency_ms` field stored at OCR time.

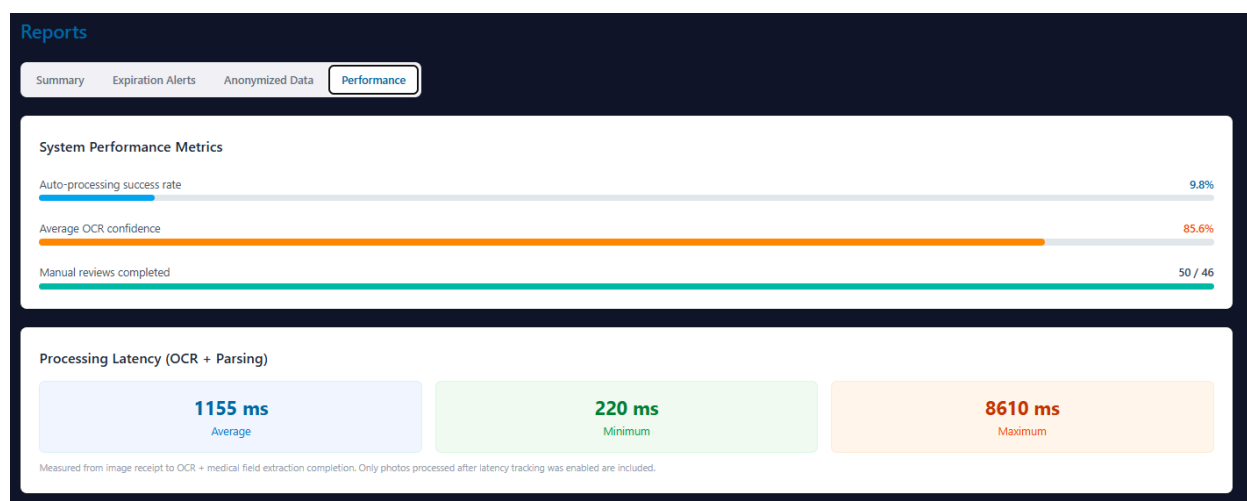


Figure 22: Performance of application

Report 5 — Control Type Distribution (Statistics Page)

Location: Statistics page → Control Type Distribution chart

What it shows: A bar chart or pie chart (toggle between both) showing how many medical checks were performed by type: Angajare (hiring check), Periodic, Adaptare, Reluare, Supraveghere, Alte. Filterable by date range.

Example from live data (last 30 days): Periodic: 3 · Angajare: 3 · Adaptare: 4 · Reluare: 4 · Supraveghere: 4 · Alte: 4

Medical relevance: Occupational health managers track the volume of each check type to plan staffing for medical units. A spike in Angajare checks signals high hiring activity; a high

Reluare count indicates employees returning after medical leave. This report supports capacity planning and regulatory compliance audits.

Implementation: The frontend fetches all photos for the selected date range from `/photos?start=&end=` and processes the `tip_control` field client-side to produce aggregated chart data. The chart type toggle switches between Recharts `BarChart` and `PieChart` components.

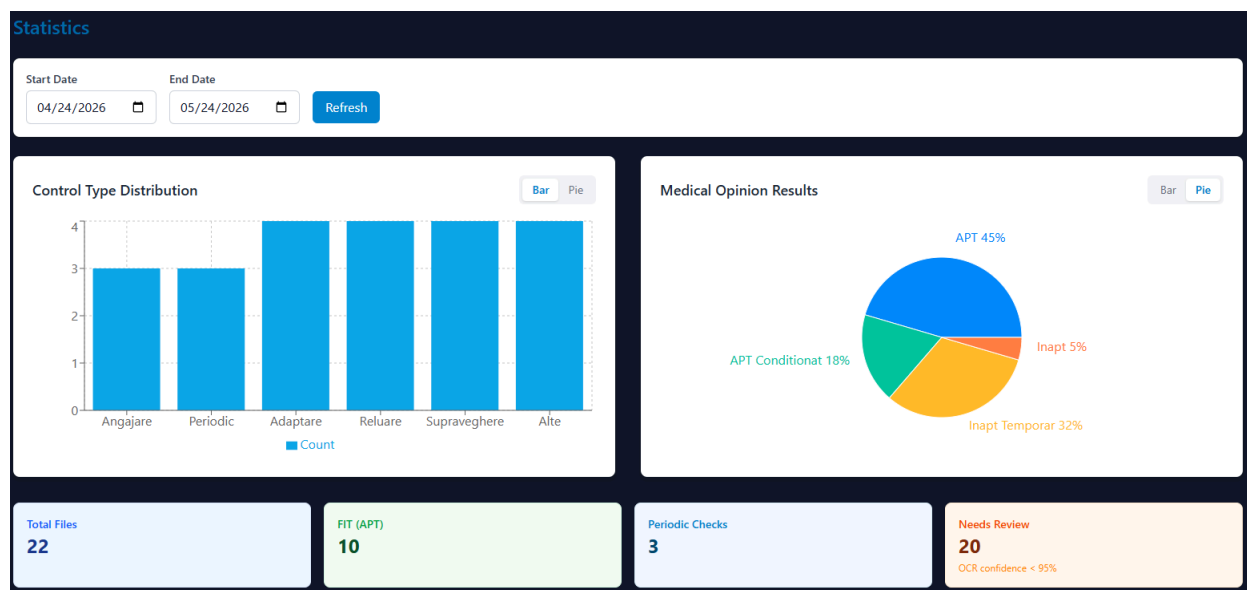


Figure 23: Statistic based off OCR and database

Report 6 — Monthly Activity Comparison

Location: Reports → Summary tab → top KPI cards

What it shows: Four key performance indicators displayed as summary cards: Total Checks (all time), Last Month count, This Month count, and Expiring Next Month count. A fifth card shows Needs Review — records where OCR confidence was below the 95% threshold.

Example from live data: Total: 51 · Last Month: 10 · This Month: 15 · Expiring Next Month: 7 · Needs Review: 46

Medical relevance: Month-over-month comparison lets managers identify trends — an increase in This Month vs Last Month may indicate a scheduled periodic review campaign. The Needs Review count is critical for data quality: records below 95% OCR confidence have

flagged fields that require a human operator to manually verify before the data is used for compliance decisions.

Implementation: The backend `/reports/summary` endpoint returns pre-computed counts using MongoDB `$facet` aggregation. Date boundaries are computed server-side using Go's `time` package to ensure timezone-consistent monthly boundaries.

Summary Table

#	Report Name	Tab / Page	Key Metric	Medical Purpose
1	Medical Opinion Breakdown	Reports → Summary	Fit/Unfit counts	Workforce fitness overview
2	Expiration Alerts	Reports → Expiration Alerts	Days until expiry	Proactive renewal scheduling
3	Anonymized Dataset	Reports → Anonymized Data	PHI-free records	GDPR-compliant research export
4	System Performance	Reports → Performance	OCR latency & confidence	Data quality monitoring
5	Control Type Distribution	Statistics	Checks by type	Capacity planning
6	Monthly Activity	Reports → Summary	Month-over-month counts	Trend analysis & compliance

OSSF Criticality Score

The OpenSSF Criticality Score calculation produced a final **original_pike_score** of **0.19619** for the repository. This result indicates that the project currently has a low criticality score from an open-source ecosystem perspective. This is expected for an academic project or a recently created repository, because the score is influenced by public repository activity and adoption indicators such as contributor count, organization count, commit frequency, issue activity, releases, and GitHub mentions.

The output shows that the repository is written mainly in Go and was created on 27 March 2026, with the latest update recorded on 24 May 2026. The repository has a small number of contributors and no public stars, releases, or large ecosystem visibility indicators. Because of this, the score remains low even though the project itself contains several implemented security features.

Therefore, the score should not be interpreted as a measure of code quality or security implementation quality. Instead, it reflects the repository's current external importance and public activity level. For this project, the low score is reasonable because the repository is new, educational, and not widely used as a public open-source dependency.

The OpenSSF Criticality Score was calculated as part of the project documentation in order to provide the default repository criticality score required. This score gives an indication of the project's importance based on repository activity and metadata signals, using the OpenSSF Criticality Score tooling and the default **original_pike.yml** configuration.

The calculation was performed in two stages. First, the repository signals were collected into a CSV file named **signals.csv**. This file contains the raw metadata used by the scoring tool, such as repository activity and project-related indicators. After the signals were generated, the default scoring configuration file, **original_pike.yml**, was downloaded from the official OpenSSF Criticality Score repository. The configuration file was downloaded using the following command:

```
curl -L -o original_pike.yml \
https://raw.githubusercontent.com/ossf/criticality_score/main/config/scorer/original_pike.yml
```

```
$ cat signals.csv
repo.url,repo.language,repo.license,repo.star_count,repo.created_at,repo.updated_at,legacy.created_since,legacy.updated_
since,legacy.contributor_count,legacy.org_count,legacy.commit_frequency,legacy.recent_release_count,legacy.updated_issue
s_count,legacy.closed_issues_count,legacy.issue_comment_frequency,legacy.github_mention_count
https://github.com/mezarnaoi/SS-project,Go,,0,2026-03-27T18:45:06Z,2026-05-24T16:00:55Z,1,0,6,0,2.62,2,0,0,0,1
```

Figure 24: Part of signals.csv file



Figure 14: Repository signals generated for Criticality Score calculation

After obtaining the configuration file, the final score was calculated by passing the existing `signals.csv` file to the `scorer` tool together with `original_pike.yml`:

```
cat signals.csv | scorer -config original_pike.yml -> score.csv
```

The command generated a new file named `score.csv`, which contains the final OpenSSF Criticality Score for the project repository. This output was used as the default criticality score result required in the documentation.

```
$ cat score.csv
repo.url,repo.language,repo.license,repo.star_count,repo.created_at,repo.updated_at,legacy.created_since,legacy.updated_
since,legacy.contributor_count,legacy.org_count,legacy.commit_frequency,legacy.recent_release_count,legacy.updated_issue
s_count,legacy.closed_issues_count,legacy.issue_comment_frequency,legacy.github_mention_count,original_pike_score
https://github.com/mezarnaoi/SS-project,Go,,0,2026-03-27T18:45:06Z,2026-05-24T16:00:55Z,1,0,6,0,2.62,2,0,0,0,1,0.19619
```

Figure 25: Generated score.csv result